Noise and Uncertainty in Molecular Property Prediction:

Robust Models and Adaptive Representations

Adelaide Punt

[Your College]

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

[Term] 2026

Statement of Originality

I, the undersigned, declare that this is my own work unless where otherwise acknowledged and referenced.

This thesis includes work carried out in collaboration. Chapters 3 and 4 are adapted from a jointly authored manuscript, Punt, A., Hanser, T., Werner, S., and Morris, G.M., in preparation for the *Journal of Cheminformatics*. A signed Statement of Authorship covering that work is included below. Chapters 6 and 7 include work carried out with Lhasa Limited and with the Oxford Protein Informatics Group.

A.P. implemented the NoiseInject framework, conducted all experiments, and drafted the manuscript. T.H., S.W., and G.M. supervised the project, contributed to the experimental design, and revised the manuscript.

Candidate: Adelaide Punt

Signed: _____

Date: _____

Acknowledgements

Write the acknowledgements. The people and bodies to thank are listed below; the funding sentence is already carried verbatim from the NoiseInject manuscript.

This research was supported by the University of Oxford's UK EPSRC Centre For Doctoral Training in Sustainable Approaches to Biomedical Science: Responsible and Reproducible Research (EP/S024093/1) and by the not-for-profit organization Lhasa Limited.

Abstract

Write the abstract (about one page): the noise problem in molecular property prediction, then one clause per chapter in order, then the open-source NoiseInject note.

Contents

Statement of Originality i

Acknowledgements ii

Abstract iii

List of Abbreviations xiii

1 Introduction 1

1.1 The questions this thesis asks . . . 3
1.2 From noise robustness to adaptive representations . . . 3
1.3 Outline of the thesis and contributions . . . 3
1.4 Collaborations and funding . . . 3

2 Background 4

2.1 Introduction . . . 4
2.2 Molecular Representations in QSAR . . . 4
2.3 Models for molecular property prediction . . . 10
2.3.1 Probabilistic decision trees . . . 12
2.3.2 Support vector machines . . . 13
2.3.3 Gaussian processes . . . 13
2.3.4 Bayesian neural networks . . . 14
2.4 Molecular similarity and activity cliffs . . . 17
2.5 Noise in molecular data . . . 17
2.5.1 Artificial noise injection . . . 19
2.6 Learning with noisy labels . . . 21
2.7 NoiseInject: a benchmarking framework, and the precompute pipeline 23
2.8 Evaluation methodology . . . 27
2.9 Conclusions . . . 29

3 Noise Robustness Across Molecular Representations and Model Architectures **31**

3.1 Introduction . . . 31
3.2 Datasets and noise injection . . . 32
3.3 Molecular representations . . . 33
3.4 Representation governs clean performance, architecture governs robustness . . . 34
3.5 Probabilistic models are no more noise-robust than deterministic ones 37
3.6 Relative model robustness is largely strategy-independent . . . 40
3.7 The pattern holds across prediction targets . . . 44
3.8 Conclusions . . . 44

4 Uncertainty Quantification Under Label Noise **48**

4.1 Introduction . . . 48
4.2 Models with an explicit noise term best track prediction error under noise . . . 48
4.3 NGBoost and the GP stay near target coverage while Bayesian variants miscalibrate . . . 50
4.4 Uncertainty attribution is itself representation-dependent . . . 50
4.5 Fingerprint representations carry the usable uncertainty signal . . . 50
4.6 Conclusions . . . 52

5 A Method for Building Adaptive Hybrid Representations **54**

5.1 Motivation: the failure of similarity-based noise handling . . . 54
5.2 The adaptive hybrid representation . . . 56
5.2.1 Review of existing approaches . . . 56
5.3 The method as a search . . . 58
5.3.1 Component representations . . . 58
5.3.2 Feature scoring and budget allocation . . . 59
5.3.3 Search configuration and internal evaluation . . . 61
5.4 Blueprints and explainability . . . 62
5.5 Default settings . . . 62
5.6 Evaluation metrics, models, and splitting . . . 62
5.7 Implementation . . . 64
5.8 Conclusions . . . 64

6 Why Hybrid Representations Help: Complexity, Explainability, and Extension **65**
6.1 Introduction . . . 65
6.2 When it helps . . . 66
6.3 Why it helps . . . 66
6.4 Explainability and stability . . . 66
6.5 Breadth across domains . . . 66
6.6 Forward compatibility: absorbing new embeddings . . . 66
6.7 Federated data (with Lhasa) . . . 67
6.8 Conclusions . . . 68

7 Molecular Similarity and Label Noise **69**
7.1 Introduction . . . 69
7.2 Similarity and the representation . . . 69
7.2.1 Distance metrics for continuous molecular descriptors . . . 69
7.2.2 Feature importance weighting . . . 70
7.3 Noise estimation and mitigation inside the method . . . 70
7.3.1 Sample removal methods . . . 70
7.3.2 Noise magnitude estimation . . . 71
7.3.3 Label smoothing and sample weighting . . . 72
7.3.4 Recovery rate . . . 73
7.4 Conclusions . . . 73

8 Discussion and Future Directions **75**
8.1 Conclusions . . . 75
8.2 Limitations . . . 75
8.3 Future directions . . . 76

Summary of Research Contributions **78**
8.4 Research Papers . . . 78
8.5 Conference Presentations . . . 78
8.6 Blogposts . . . 78

A Datasets and curation **80**

B NoiseInject supplementary material **81**
B.1 Default hyperparameters for all models evaluated in this study . . . 81

B.2 Pairwise Spearman rank correlations between model NDS profiles . . 81
B.3 Pairwise Spearman rank correlations between representation NDS profiles 81
B.4 Intraclass correlation coefficients (ICC(1,1)) for model pairs 82
B.5 Configurations excluded from robustness analysis 82
B.6 Global overview of noise robustness on the ECFP4 representation . . 82
B.7 Baseline R^2 versus NDS scatter (PDV, Gaussian) 82
B.8 Neural-network Bayesian transformation effects by representation . . 82
B.9 Uncertainty quantification metrics for probabilistic models on the ECFP4 representation 83
B.10 ANOVA variance decomposition on external validation datasets . . . 83
B.11 RF vs QRF on validation datasets 83
B.12 Default hyperparameters used for external validation experiments . . 83

C The search space and its defaults **84**

D Extended result tables **85**

E Data and code availability **86**

List of Figures

2.1 Visualisation of the molecule O=C1C=CN(C1)C=O using three different methods: a) ECFP4 showing the molecule broken into substructures with circular neighbourhoods for each atom. Each substructure is hashed to produce unique identifiers, which are then compiled into a binary bit vector representing the presence or absence of specific substructures. b) SMILES tokenization and one-hot encoding of the molecule. The SMILES string is tokenized, and using a vocabulary of the most common tokens, these tokens are converted into a one-hot encoded matrix. c) Molecular graph representation where nodes represent atoms and edges represent bonds. 5

2.2 Deterministic versus Bayesian neural network architectures. Deterministic networks use fixed weights W to produce point predictions, while Bayesian networks treat weights as distributions $\mathcal{N}(\mu, \sigma^2)$, generating prediction histograms through multiple stochastic forward passes to quantify epistemic uncertainty. 16

2.3 Effect of each noise injection strategy on the QM9 HOMO–LUMO gap label distribution (blue) at $\sigma = 0.5$. The RMSE annotated in each panel is computed between the clean and the noisy labels for that strategy (i.e. the magnitude of perturbation). a) Gaussian (red); b) Outlier (brown); c) Quantile (green); d) Threshold (purple); e) Value-Proportional (orange); and f) Heteroscedastic (pink). 22

2.4 Memory-mapped file architecture contrasted with traditional in-memory loading. Top: CSV/Parquet approaches load entire datasets into RAM, creating bottlenecks at scale. Bottom: Memory-mapped binary files enable on-demand line-by-line access with constant low RAM usage. Right: Entry-level structure showing byte-aligned fields with fixed-size representations (target values, fingerprints, domain labels) and length-prefixed variable-length fields (SMILES strings, one-hot encodings). 25

3.1 ANOVA variance decomposition (η^2, %) for a) predictive performance (R^2 at $\sigma = 0.3$) and b) noise robustness, quantified by the noise degradation slope (NDS), by noise strategy. Lower (more negative) NDS indicates faster performance loss under noise. 36

3.2 Representation–model interaction effects on the QM9 HOMO–LUMO gap. a) Noise degradation slope (NDS) heatmap showing model robustness across representations under Gaussian noise; lower (more negative) NDS indicates faster degradation. "N/A" indicates configurations excluded due to baseline $R^2 < 0.6$ or kernel incompatibility (GP uses a Tanimoto kernel defined on binary vectors, which is incompatible with PDV). b) NDS on PDV versus ECFP4 fingerprint for each model under Gaussian noise, with Spearman correlation. . . 38

3.3 Global overview of noise robustness on the QM9 HOMO–LUMO gap. a) Performance degradation curves (PDV, Gaussian noise) showing R^2 versus noise level σ for representative models. b) Noise degradation slope (NDS) heatmap across all model architectures and noise strategies on PDV; lower (more negative) NDS indicates faster degradation under noise. The Tanimoto GP is absent because its kernel is incompatible with PDV (see Section 2); a separate GP with an RBF kernel is included for PDV-specific analyses. 39

3.4 Neural network family comparison on the QM9 HOMO–LUMO gap with N=10k: R^2 versus σ under Gaussian noise on the physicochemical descriptor vector (PDV) representation for a) NN-α family, b) NN-β family, and c) RF versus QRF. Heteroscedastic results (not shown) are qualitatively consistent. 41

3.5 Noise degradation slope (NDS) heatmaps for three external validation datasets: a) LogD, b) Caco-2 Efflux, and c) hERG K_i. Lower (more negative) NDS indicates faster degradation under noise. Black cells indicate baseline $R^2 < 0.3$ for that (model, strategy) pair; "N/A" indicates filtered extreme values ($|NDS| > 2$). Models whose baseline R^2 fell below 0.3 across all strategies are omitted entirely, accounting for the smaller set of rows on hERG K_i. 45

3.6 Comparison of noise robustness on QM9 (HOMO–LUMO gap, N = 10,000) and three external validation datasets: LogD (N = 5,039), Caco-2 Efflux (N = 2,161), and hERG K_i (N = 1,482). a) Model robustness measured by the noise degradation slope (NDS) across all four datasets; lower (more negative) NDS indicates faster degradation under noise. b) QM9 vs external robustness: models with inherent robustness mechanisms (NGBoost, SVM) maintain their relative standing, while ensemble-dependent models (XGBoost) degrade on the external datasets. 46

4.1 Uncertainty estimates under label noise on the QM9 HOMO–LUMO gap. a) Mean predicted uncertainty versus σ for probabilistic models (PDV, Gaussian strategy). b) Aleatoric and epistemic uncertainty components versus σ for VBLL models. 51

List of Tables

2.1 Regression noise injection strategies. For each point i, noise $\varepsilon_i \sim \mathcal{N}(0, s_i^2)$ is added to the target y_i, where the scale s_i is defined per row. Here $z_i = (y_i - \bar{y})/\sigma_y$ is the standardised target and Q_{10}, Q_{90} are the 10th and 90th percentiles of the target distribution. 20

2.2 Summary of metrics used in this study, their definitions, and their intended interpretation. $\hat{y}_i$ and $\hat{u}_i$ denote the predicted value and predicted uncertainty for sample i; σ is the noise scaling factor (Table 2.1). 30

3.1 ANOVA variance decomposition by noise strategy on the QM9 HOMO–LUMO gap. Each cell reports η^2 (%), the share of variance explained by that factor. Columns: **Model** = model architecture effect; **Rep** = molecular representation effect; **Inter.** = model×representation interaction; **Resid.** = residual unexplained variance. "Performance" uses R^2 at $\sigma = 0.3$ as the outcome; "Robustness" uses the noise degradation slope (NDS). Bold entries highlight the dominant factor in each row. 35

3.2 Wilcoxon signed-rank tests for Bayesian and probabilistic transformations on the QM9 HOMO–LUMO gap. **Base** is the deterministic reference model; **Variant** is the probabilistic/Bayesian transformation tested against it. **Δ NDS** = mean change in noise degradation slope (NDS) from base to variant; positive values mean the variant is more robust (slope closer to zero). **Sig.** marks pairs that are statistically significant at $p < 0.05$ (asterisk). 40

3.3 Noise degradation slope (NDS) by model on PDV (QM9 HOMO–LUMO gap), ranked by mean across six strategies. Lower |NDS| (slope closer to zero) indicates greater robustness. Strategy abbreviations: **Gauss.** = Gaussian, **Quant.** = Quantile, **Thresh.** = Threshold, **Hetero.** = Heteroscedastic, **Val.-P.** = Value-Proportional. Bold rows mark the two most robust models; bold cell entries mark each strategy's most robust configuration. 42

4.1 Strongest and weakest model–representation combinations for uncertainty–noise correlation on the QM9 HOMO–LUMO gap (Gaussian noise strategy). Columns: **Unc-Noise** ρ = Spearman correlation between predicted uncertainty and injected noise magnitude $|\epsilon|$ (higher = better noise detection); **Unc-Error** ρ = Spearman correlation between predicted uncertainty and absolute prediction error; **ECE** = expected calibration error (lower = better-calibrated); **Cov.** 1σ / 2σ = empirical coverage at the 1σ (68% target) and 2σ (95% target) intervals. . 49

List of Abbreviations

ADME Absorption, distribution, metabolism, and excretion

ANOVA Analysis of variance

BNN Bayesian neural network

CP Conformal prediction

DFT Density functional theory

ECE Expected calibration error

ECFP Extended-connectivity fingerprint

ECFP4 Extended-connectivity fingerprint with radius 2, diameter 4

F1 F1 score (harmonic mean of precision and recall)

GCN Graph convolutional network

GIN Graph isomorphism network

GNN Graph neural network

GP Gaussian process

HOMO Highest occupied molecular orbital

ICC Intraclass correlation coefficient

LightGBM Light Gradient Boosting Machine

LNL Learning with noisy labels

LUMO Lowest unoccupied molecular orbital

MAE Mean absolute error

MHG-GNN Molecular Hypergraph Grammar Graph Neural Network

ML Machine learning

MLP Multi-layer perceptron

mRMR Minimum redundancy, maximum relevance

NDS Noise degradation slope

NGBoost Natural gradient boosting

NN Neural network

NN-α Neural network architecture α ([128, 64], dropout after each layer)

NN-β Neural network architecture β ([128, 128], dropout before output)

OHE One-hot encoding

PCA Principal component analysis

PDV Physicochemical descriptor vector

QRF Quantile regression forest

QSAR Quantitative structure–activity relationship

QSPR Quantitative structure–property relationship

RBF Radial basis function

RF Random forest

RFE Recursive feature elimination

RMSE Root mean squared error

ROC-AUC Area under the receiver operating characteristic curve

SHAP SHapley Additive exPlanations

SMILES Simplified molecular input line entry system

SNS Sort & Slice

SVM Support vector machine

VBLL Variational Bayesian Last Layer

XGBoost eXtreme Gradient Boosting

Chapter 1

Introduction

New avenues in the drug discovery pipeline have opened up thanks to advances in large data set collection, analysis, and computation. Therapeutic targets are identified, potential drug candidates are screened and optimized, and the most promising of these go on to pre-clinical and clinical trials [Lill, 2007]. There are various methods by which potential drug candidates can be screened. Virtual screening allows for the rapid and large-scale analysis of libraries of compounds, and is a particularly useful approach used to speed up the drug discovery process [Neves et al., 2018]. One standard screening method is the prediction of bioactivities and pharmacological properties of such candidates. There is a long history of using Quantitative Structure-Activity Relationship (QSAR) models to make these predictions, to act as a filter for compounds with undesirable properties, and thus reduce the time and cost associated with experimental screening [Sabando et al., 2022]. QSAR models aim to predict the relationship between a molecule's molecular features as encoded within a particular molecular representation and a desired molecular property [Sabando et al., 2022] of interest. A QSAR model typically consists of a molecular representation paired with a machine learning (ML) model. An ideal molecular representation will allow a model to learn only the relevant information about each molecule with respect to the target property. Representations are either 'static' hand-crafted descriptors or circular fingerprints that encode predefined structural and physicochemical features [Rogers and Hahn, 2010, David et al., 2020, Dablander et al., 2024]; or learned representations, where the encoding is derived from a predictive model, often taken from the activations of the final layer of a neural network, or a latent space embedding [Sabando et al., 2022, Kishimoto et al., 2023].

QSAR models are of course, sensitive to the quality of the input data on which

they are trained. Experimental noise can arise from measurement errors, variability under experimental conditions, varying provenance of the data, and intrinsic biological and chemical variability. Even the very measures of model performance may be flawed if test and validation sets are laden with noise. Kolmar and Grulke [2021] show that the variance associated with experimental data can contribute more to prediction error than the model's own error. There is also a limit to how much noise the model can tolerate; Jorner et al. [2021] found that once the model error approaches the magnitude of the experimental measurement error on labels, adding more training data yields diminishing returns in prediction accuracy. In the context of activity prediction, the inherent variability within experimental setups and assays, batch effects, measurement errors, and biological variability introduce additional layers of complexity [Kolmar and Grulke, 2021, Fourches et al., 2010]. All of this increases the likelihood of encountering noise in the data [Fang and Ye, 2022]. Even the molecular representations in the training data themselves may be incorrect or misleading [Walters, 2023].

This thesis addresses the challenge of conducting molecular property prediction when the underlying experimental data contain noisy labels. My initial research question is which combinations of molecular representations and predictive model architectures maintain predictive performance under label corruption, and whether probabilistic methods that quantify uncertainty justify their additional complexity. As I will show, representation choice dominates predictive performance on clean data, but this relationship inverts under label corruption—predictive architecture becomes the primary factor determining robustness. Building on these findings, I have begun exploring noise mitigation techniques, leveraging identified noise-uncertainty correlations and developing novel methodologies for noisy settings. To enable systematic evaluation at this scale, I developed a custom computational pipeline supporting high-throughput experimentation, extensive replication across large datasets, and comprehensive testing of hundreds of configurations while maintaining memory efficiency through optimized storage strategies.

Most existing work on noise in QSAR assumes homoscedastic Gaussian label noise and evaluates either molecular representations or machine learning methods in isolation. Cortes-Ciriano et al. [2015] examined 12 algorithms across multiple noise levels but used fixed representations. Studies that compare representations are typically done on clean data. No work has systematically compared the relative importance of representation versus architecture choice for noise robustness,

assessed whether probabilistic methods justify their computational overhead in noisy settings, or tested whether findings generalize across different corruption patterns and molecular property types. This thesis fills that gap. In addition, I will discuss noise mitigation, using the results of the first half of the thesis. We explore why many noise mitigation techniques underperform in cheminformatics data and methods of resolution in the form of novel molecular representations.

1.1 The questions this thesis asks

This research addresses three main aspects of the robustness of QSAR models under label noise. First, we investigate the contributions of molecular representation and model architecture on both overall predictive performance and, more specifically, noise robustness. Second, we compare how noise-robust probabilistic models perform versus their deterministic counterparts, and whether their uncertainty estimates correlate with prediction error or label noise under noisy conditions. Third, we assess the generalizability of noise-robustness patterns across different noise-injection mechanisms and molecular properties. Together, these questions aim to identify what, if anything, makes a QSAR model robust to noise. Finally, we introduce NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty quantification in machine learning models.

1.2 From noise robustness to adaptive representations

Write Sec 1.4 from the outline bullets above. The raw material is Sec 5.1.

1.3 Outline of the thesis and contributions

Write Sec 1.5 from the outline bullets above.

1.4 Collaborations and funding

Write Sec 1.6.

Chapter 2

Background

2.1 Introduction

I will now break down the two components of QSAR models: molecular representations and predictive models:

2.2 Molecular Representations in QSAR

The success of machine learning hinges on the quality of input data. Although the quality can refer to noise, which the majority of my work is centered around, in this case I'm talking about the type of input labels. Molecular representations provide different features with which QSAR models and noise detection techniques can be learned. However, a potential bottleneck on larger data sets should be noted; the more complex the data structure, the greater the computational cost. For smaller data sets, the number of parameters is limited by the size of the training set, to prevent overfitting.

One of the most common and intuitive molecular representations are Simplified Molecular-Input Line-Entry System (SMILES) strings which consist of a sequence of ASCII characters generated by a fixed set of rules to specify a given chemical identity [Weininger, 1988]. A common way to use SMILES strings as inputs to QSAR models is to tokenize and one-hot-encode them rather than to use the strings themselves [Backenköhler et al., 2023]. The strings are tokenized into atomic branches and symbols and mapped to indices of a vocabulary, and finally converted into binary vectors using one-hot encodings [Backenköhler et al., 2023]. This representation, referred to as one-hot-encoded SMILES, allows models to process molecular structures

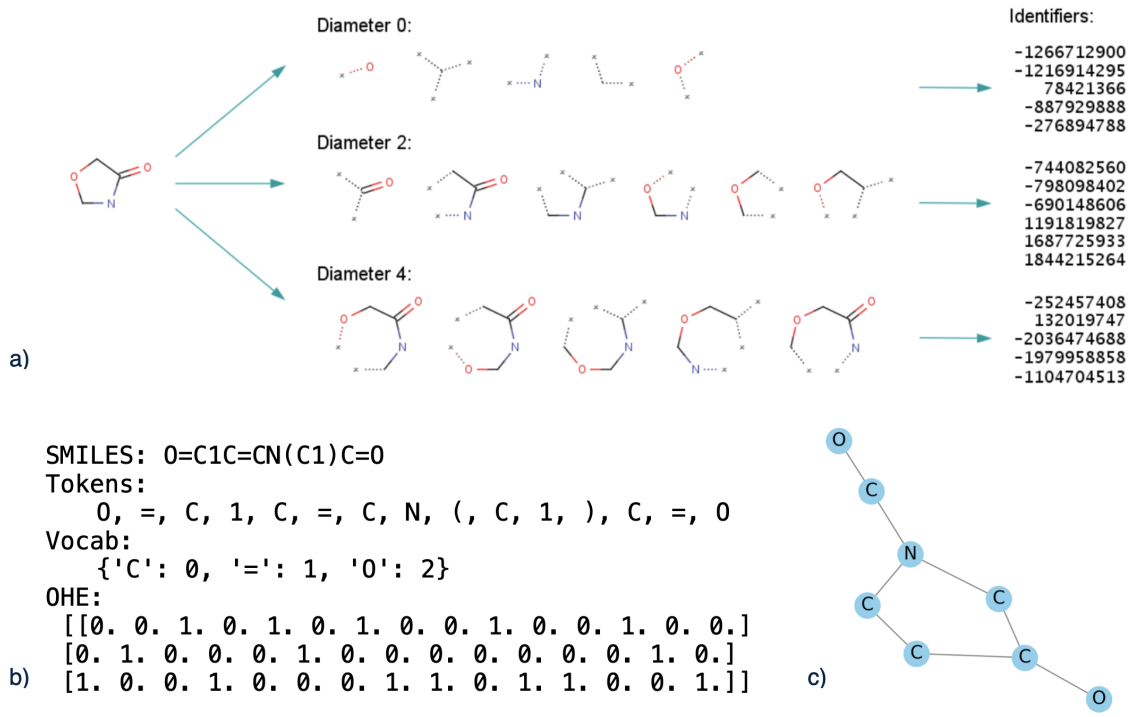


Figure 2.1: Visualisation of the molecule O=C1C=CN(C1)C=O using three different methods: a) ECFP4 showing the molecule broken into substructures with circular neighbourhoods for each atom. Each substructure is hashed to produce unique identifiers, which are then compiled into a binary bit vector representing the presence or absence of specific substructures. b) SMILES tokenization and one-hot encoding of the molecule. The SMILES string is tokenized, and using a vocabulary of the most common tokens, these tokens are converted into a one-hot encoded matrix. c) Molecular graph representation where nodes represent atoms and edges represent bonds.

as bit-vector-based fingerprints or as sequences of vectors. Sequence-based models such as recurrent neural networks and transformers have proven to be particularly successful in capturing text semantics within a sequence. When applied to raw or one hot encoded (OHE) SMILES strings, these models can effectively capture the structure of the molecule in its textual representation [Backenköhler et al., 2023].

Another common representation is the molecular fingerprint, a vector containing information about the properties of the molecule in question. In a typical bit vector-based fingerprint, each bit accounts for the presence (1) or absence (0) of a given chemical feature or substructure [Fassio et al., 2022]. One common fingerprint is the

Molecular ACCess System (MACCS) key, a substructure key-based fingerprint that encodes predefined atom symbols and properties, as well as bond types [Xie et al., 2020]. Another is the Extended Connectivity Fingerprint (ECFP), which encodes local neighborhoods around each atom and bond connectivity in molecules [Xie et al., 2020]. ECFPs are typically constructed with a radius of 2, a length of 2048 bits, and active chirality flags. These existing molecular representations are well established and have been quite successful when coupled with QSAR models [Cortés-Ciriano et al., 2020, Sabando et al., 2022]. They are relatively simple, and as the bits can be mapped back to one or more substructures, they are human-interpretable, and furthermore easy-to-compute. However, they are subject to information loss due to bit collisions, which happens when two different substructures are encoded at the same bit position [Fassio et al., 2022]. They are also based on a fixed procedure and are not trainable or adaptable to a particular dataset. Alternative representations that capture different structural, chemical, or physical features are able to overcome these limitations.

Recent advances have led to the development of new molecular fingerprints. One such method is the Sort & Slice (SNS) technique, an alternative to hash-based folding for pooling ECFP substructures [Dablander et al., 2024]. Sort & Slice begins by sorting all circular substructures in the training set according to their frequency of occurrence. It then "slices" away all but the N most frequent substructures, which are subsequently used to generate an N-dimensional, bit-collision-free binary fingerprint. This approach solves the ambiguity problem of hashed fingerprints, where multiple substructures might be encoded at the same bit position, making it very interpretable [Dablander et al., 2024]. Sort & Slice fingerprints outperform ECFP4 with standard hash-based folding methods across various molecular property prediction tasks. These experiments consisted of different data splitting techniques, machine learning models, fingerprint diameters, fingerprint dimensions, and atomic invariants [Dablander et al., 2024]. Furthermore, Sort & Slice outperformed advanced substructure selection schemes such as filtered fingerprints and mutual information maximization [Dablander et al., 2024].

Neural fingerprints are a category of learned representations that overcome the fixed, hard-coded nature of traditional fingerprints. Unlike ECFP or MACCS keys, which rely on predefined hashing schemes or substructure catalogues, neural fingerprints train on molecular data to capture chemical information [Jaeger et al., 2018a, Chithrananda et al., 2020]. These representations are the result of deep learning

architectures that have learned dense, continuous embeddings which can be used for downstream tasks.

Mol2vec, which adapts the Word2vec algorithm from NLP to molecular substructures [Jaeger et al., 2018a], is common neural fingerprint. Mol2vec treats molecules like "sentences" and circular substructures derived from Morgan fingerprints like "words". The model learns vector embeddings for each substructure through unsupervised training, such that chemically similar substructures are mapped to nearby points in the embedding space [Jaeger et al., 2018a]. The embeddings of all present substructures are summed to obtain a full representation. This approach produces 300-dimensional dense vectors that avoid the sparsity and bit collisions inherent in hashed fingerprints [Jaeger et al., 2018a]. Mol2vec has performed strongly on various QSAR benchmarks, matching or exceeding Morgan fingerprints on solubility and bioactivity prediction tasks [Jaeger et al., 2018a].

Transformer-based architectures have also been applied to molecular representation learning, similarly stemming from NLPs ideas. MolBERT applies a bidirectional transformer to learn molecular representations from SMILES strings through self-supervised pre-training [Fabian et al., 2020]. The model has domain-relevant auxiliary adaptations, such as the prediction of calculated physicochemical properties alongside the masked token prediction, to improve the chemical fidelity of the learned embeddings [Fabian et al., 2020]. MolBERT demonstrated improvements over traditional fingerprints and graph-based methods on Virtual Screening and QSAR benchmarks [Fabian et al., 2020]. ChemBERTa is another transformer-based approach which pre-trains masked language models on SMILES data from PubChem [Chithrananda et al., 2020]. ChemBERTa uses a tokenization strategy adapted for the SMILES notation, which allows the model to learn representations of atoms and functional groups through self-supervised learning [Chithrananda et al., 2020]. Attention visualization reveals that ChemBERTa learns chemically meaningful patterns, with different attention heads specializing in functional groups, ring structures, or SMILES syntax [Chithrananda et al., 2020]. ChemBERTa-2, an optimized variant pre-trained in 77 million compounds, achieved even stronger performance than the original ChemBERTa in MoleculeNet benchmarks [Ahmad et al., 2022].

Neural fingerprints provide several advantages over traditional fingerprints. They produce dense continuous vectors that avoid sparseness and information loss from bit collisions [Jaeger et al., 2018a]. They can also be pre-trained once on large unlabeled datasets and then fine-tuned for specific downstream tasks, opening the

door to transfer learning [Chithrananda et al., 2020, Fabian et al., 2020]. However, these methods are often computationally expensive [Jaeger et al., 2018a] and due to their black-box nature, they are not as interpretable. [Chithrananda et al., 2020]. However, attention mechanisms can have some degree of interpretability through visualization [Chithrananda et al., 2020].

Beyond fingerprints, physicochemical-descriptor vectors (PDVs) are a class of continuous-value molecular descriptors that encode structural and chemical properties of molecules [Todeschini and Consonni, 2009]. Unlike binary fingerprints or graph-based representations, PDVs aggregate atom and bond-level quantities into fixed-length feature vectors that capture a wide variety of information about a molecule and its structure [Mauri et al., 2006]. PDVs typically consist of hundreds of properties including constitutional descriptors such as molecular weight, atom and bond counts, and heteroatom ratios; topological indices encoding molecular connectivity patterns; physicochemical properties such as logP (lipophilicity), molar refractivity, and topological polar surface area; and electronic descriptors derived from partial charges and polarizabilities [Todeschini and Consonni, 2009, Cherkasov et al., 2014]. The `RDKit` cheminformatics toolkit provides a comprehensive suite of more than 200 molecular descriptors that cover these categories [Landrum]. PDVs are very interpretable; each descriptor has a clear physicochemical meaning [Cherkasov et al., 2014]. They are also less computationally expensive than some 3D methods or neural fingerprints, with many descriptors simply requiring a 2D molecular structure without conformer generation or extensive pre-training. This efficiency makes PDVs particularly appropriate for high-throughput applications. However, it is also possible to use descriptors based on 3D components [Mauri et al., 2006].

However, PDVs have limitations. The dimensionality of PDV feature spaces can lead to problems with multicollinearity [Todeschini and Consonni, 2009]. This can be mitigated by feature selection or dimensionality reduction techniques [Cherkasov et al., 2014]. Additionally, PDVs encode molecular properties as aggregated global features, which may not capture fine-grained structural details such as specific substructure arrangements or local chemical environments, information important for particular endpoints [Mauri et al., 2006].

One of the more intuitive representations are molecular graphs. These graphs can be categorized into 2D and 3D graphs. 2D graphs rely on adjacency matrices that encode topological atom- and bond-level features but leave out geometric properties such as bond angles or torsion, meaning they may fail to capture stereochemistry.

3D graphs incorporate these spatial features, but are often limited by remaining in a single conformation [Guo et al., 2023]. A standard 3D molecular graph is constructed such that each molecule is represented as $G = (V, E)$ with node features $\mathbf{x}_i$ and edge features $\mathbf{e}_{ij}$. Nodes encode atom type, degree, hybridization, aromaticity, and formal charge; edges encode bond order and conjugation [Guo et al., 2023].

Another common molecular graph is the standard distance compatibility molecular graph, in which atoms and bonds are mapped to sets of nodes and edges [David et al., 2020, Hönig et al., 2023]. These molecular graphs are typically 2D objects that represent 3D information such as atomic coordinate, bond angles, chirality, etc. [David et al., 2020]. However, there are alternative molecular graph representations such as graphs with quantum 3D embeddings, weight alignment graphs, association graphs containing pairs of molecules, and geometric interaction graph neural network that incorporate 3D structures and physical interactions [Hönig et al., 2023, Yan et al., 2023, Yang and Li, 2023]. Du et al. [2022] developed hybrid 2D and 3D graphs, combining 3D data, guaranteeing precision and completeness by providing unabridged information, and 2D data, providing chemical intuitions for guidance, in a zipper-like arrangement. These graphs pose various challenges, such as the atoms being localized at voxels creating issues with the points being too sparse, leading to poor performance. Kuzminykh et al. [2018] developed a novel approach in which atoms are extended to fill nearby voxels with a wave transformation.

These graph representations do not highlight specific substructures or specific arrangements of atoms and bonds within each molecule. The substructures contain functional groups that are related to a range of chemical properties and are potentially involved in downstream tasks related to the function of molecules. Bouritsas et al. [2023] developed graph substructure networks, a topologically aware message passing scheme based on substructure encoding. The method involves encoding structural roles based on the appearance of specific substructures and using them in message passing updates. Since message-passing GNNs are at most as powerful as the Weissfeiler Leman test, they cannot isolate sufficient features from the graph structure [Bouritsas et al., 2023]. Graph substructure networks, on the other hand, capture these substructure-related features. And recent research has highlighted the abilities of pretrained GNN encoders produce embeddings on unlabeled datasets that capture structural features [Kishimoto et al., 2023, Ross et al., 2022].

Although numerous studies have explored the effects of noise on ML algorithms for molecular property prediction, comparisons between different molecular repre-

sentations remain limited. Kolmar and Grulke [2021] examined the predictive limits of QSAR in the presence of noise, showing that by learning the underlying trends rather than fitting to noise, the models can achieve predictions more accurate than those of their training data. They assumed experimental noise could be modeled as Gaussian-distributed, using the Central Limit Theorem to argue that aggregated experimental measurements from multiple independent error sources exhibit a Gaussian distribution. However, Heid et al. [2023] used mean-variance estimation and bias-variance decomposition to show that noise can be tied to specific modalities, for example structure dependence. As an example, nitrogen-containing functional groups introduce measurement deviations, implying the presence of systematic noise.

2.3 Models for molecular property prediction

To understand model behavior in the presence of label noise, we explored a wide range of model architectures. One key issue with QSAR models is the standard assumption that labels represent true values. Most chemical data sets rely on experimentally-derived labels and lack sufficient replications to provide a robust statistical representation of the underlying distributions [Kolmar and Grulke, 2021]. A single measurement, or even multiple measurements for a given label, do not guarantee that a given label accurately reflects the population mean [Kolmar and Grulke, 2021]. Most ML models rely on the assumption that the training data is, in fact, an accurate reflection of the population. Therefore, they treat these potentially noisy labels as discrete quantities rather than distributions, which opens the door to overfitting to noise and giving unduly optimistic assessments of the models. All hyperparameters for the models below are included in Appendix B.1.

There are a wide variety of model architectures that can be used for QSAR modeling. Starting off with simpler and more explainable models, we have tree-based methods. Decision trees are non-parametric supervised learning algorithms that perform recursive binary partitioning of the feature space [Breiman et al., 1984]. A decision tree builds a predictive model by learning a series of if-then rules that recursively split the data based on given feature values, creating a tree where internal nodes represent decision rules, branches represent the results of those decisions, and leaf nodes contain predicted values or class labels [Breiman et al., 1984]. Each recursive split is chosen to maximize homogeneity within the resulting partitions. For classification, Gini impurity is commonly used and for regression, mean squared

error reduction [Breiman et al., 1984].

There are several properties that make decision trees a good foundation for QSAR modeling. First, they handle non-linear relationships between molecular descriptors and target properties without requiring explicit feature engineering or transformation [Breiman et al., 1984]. They also do not make assumptions about the underlying distribution of the data, making them robust to violations of parametric assumptions that can limit linear regression or other classical methods [Svetnik et al., 2003].

Decision trees handle data in a "contributory" rather than "consensus" manner. In consensus methods such as linear regression or neural networks, every data point influences the prediction at every other point in the feature space. Tree-based methods make predictions based on the subset of training samples that fall into the same leaf node as the test point [Breiman et al., 1984]. This mechanism essentially quarantines outliers in their respective leaf nodes, limiting their impact on predictions for samples in other regions or chemical domains.

Individual decision trees tend to have high variance and often overfit, particularly at large depths [Breiman et al., 1984]. Small modifications made to the training data can result in large differences in the structure of the tree, resulting in unstable predictions [Breiman, 2001]. Ensemble methods are used to counteract this instability, aggregating predictions from many trees. Random forests (RFs) aggregate predictions from trees trained on bootstrap samples of data with random subsets of features at each split [Breiman, 2001]. Gradient boosting methods such as XGBoost sequentially fit trees to the residuals of previous iterations, progressively reducing the prediction error [Chen and Guestrin, 2016]. Bootstrapping ensures that each tree is trained on a slightly different subset of data, which effectively reduce the influence of any individual sample [Breiman, 2001]. This is especially beneficial with noisy samples. The selection of random features at every node decorrelates the trees, preventing systematic biases from propagating through the ensemble [Breiman, 2001]. The aggregation of trees further prevents overfitting to noise by smoothing out prediction errors from individual noisy labels.

Neural networks (NNs), a very popular family of models, can also be used for QSAR, although with mixed results. A NN is a system of interconnected nodes which learn by passing information through it's layers. There are weights and biases attached to the nodes and various other parameters which the model optimizes to minimize the difference between the predicted output and the actual target value.

During training, the network parameters are updated using backpropagation, in which gradients of the loss function with respect to each weight are propagated backward through the network [Hornik et al., 1989]. The structure of these networks and the hyperparameters used depend on the type of NN. In QSAR, one of the most commonly used architectures is the multilayer perceptron (MLP), a NN containing fully-connected layers with non-linear activation functions which transforms molecular feature vectors (i.e., fingerprints, descriptors) into a property or activity prediction [Ma et al., 2015].

For molecular property prediction, it is recommended that architectures use 2-3 hidden layers with 128-512 units each, ReLU activations, and dropout regularization to prevent overfitting. Training is performed using backpropagation with gradient descent optimizers such as Adam, to minimize a loss function (MSE for regression, cross-entropy for classification) on batches of training data [Wu et al., 2018].

However, NN performance on QSAR-related tasks has been very mixed. Their complexity and use of gradients make them prone to label memorization [Zhang et al., 2017], which in the context of noise is problematic, as it means that models may memorize the noise themselves. Although models that are memorizing labels may initially appear to have strong predictive performance, they cannot generalize out of distribution samples properly and often provide inaccurate uncertainty estimates. Various approaches have been proposed to improve neural network robustness to label noise, including noise-robust loss functions [Zhang et al., 2017], confident learning to identify mislabelled examples [Northcutt et al., 2021a], and regularization techniques such as mixup or label smoothing [Zhang et al., 2017].

All the models up to this point have assumed a vector-based input. Molecular graphs, on the other hand, require different types of predict modeling. Two standard ML methods on molecular graphs include Convolutional Networks (GCNs) and Graph Isomorphism Networks (GINs). Graph Convolutional Networks (GCNs) apply the concept of convolution to graph-structured data. Instead of operating on grid-based data, such as images, GCNs aggregate information from neighbors of a node to update its representation [Backenköhler et al., 2023, Xu et al., 2019]. In contrast, Graph Isomorphism Networks (GINs) address the limitations in GCNs' ability to differentiate between non-isomorphic graphs. Unlike GCNs, which normalize and weight the sum of neighboring node features, GINs use a simple sum followed by a trainable MLP, giving the model flexibility to learn complex transformations of node features. The GIN model's design is informed by the Weisfeiler-Lehman

test for graph isomorphism, enabling it to distinguish between molecular graphs that are structurally similar but differ in subtle ways [Backenköhler et al., 2023, Xu et al., 2019]. This makes GINs particularly suited to Activity Cliffs (ACs), where small variations in molecular structure can lead to significant changes in chemical or biological activity [Backenköhler et al., 2023].

2.3.1 Probabilistic decision trees

Random Forests (RFs) are one of the most common choices for QSAR modeling thanks to their robustness and interpretability [Svetnik et al., 2003]. During bootstrapping, each tree trains on a separate subset of data, reducing the influence of any one individual label. This mechanism is particularly useful when working with noisy labels, particularly outlier noise [Breiman, 2001]. Quantile Regression Forests (QRFs) extend RF predictions to full distributions by keeping the distribution of training labels in each leaf and computing quantiles across all trees for a given prediction [Meinshausen, 2006]. It inherently accounts for heteroscedasticity, such that the quantiles will reflect the spread of a target variable's domain. We also used eXtreme Gradient Boosting (XGBoost) [Babajide Mustapha and Saeed, 2016, Tian et al., 2022], Light Gradient-Boosting Machine (LightGBM) [Ke et al., 2017], and Natural Gradient Boosting (NGBoost) [Duan et al., 2020]. NGBoost extends deterministic gradient boosting by treating the parameters of a chosen parametric distribution as regression targets and learns them via boosting with a natural gradient update rule [Duan et al., 2020]. Computationally, NGBoost scales like standard boosting in N but with larger constants that depend on the number of distributional parameters p: each iteration fits p learner series and inverts N matrices of size $p \times p$, adding $O(Np^3)$ cost per iteration [Duan et al., 2020]. Even with a two-parameter Gaussian distribution, this additional cost is non-trivial and may limit scalability to larger datasets.

2.3.2 Support vector machines

Support Vector Machines (SVMs) are a well-established baseline in QSAR modeling, mapping inputs into a high-dimensional feature space where a maximum-margin hyperplane separates predictions [Vapnik, 1995, Svetnik et al., 2003]. A radial basis function (RBF) with $C = 1.0$ and $\gamma =$ `scale` was used as the kernel.

2.3.3 Gaussian processes

One ML method that has had particular success in molecular property prediction is Gaussian Processes (GPs) [Obrezanova et al., 2007, Griffiths and et al., 2022]. A GP is a non-parametric model that outputs a Gaussian predictive distribution over every data point, providing an uncertainty estimate for each prediction [Obrezanova et al., 2007]. GPs have been known to match or exceed the predictive performance of traditional QSAR models such as support vector machines (SVMs) or partial least squares (PLS), while also providing insight into the reliability of those predictions [Obrezanova et al., 2007]. The GP defines a distribution over functions, whose structure is determined by the kernel. These kernels can be modified to fit specific domains or functionalities. Typically, a Tanimoto kernel is used when working with molecular fingerprints [Ralaivola et al., 2005, Moss and Griffiths, 2020, Griffiths and et al., 2022]. One of the major limitations to using GPs in practice is computational scaling. Inference has $O(N^3)$ time complexity and $O(N^2)$ memory, where N is the number of training points [Rasmussen and Williams, 2005]. This can be mitigated with sparse GP approximations, but these introduce approximation error and can be more difficult to tune [Quiñonero-Candela and Rasmussen, 2005]. Ensembles of trees or BNNs are typically more efficient for larger data sizes [Lakshminarayanan et al., 2017].

We also employed GP models using the `Gauche` framework [Griffiths and et al., 2022], which is optimized for cheminformatics tasks. Standardized kernels for GPs are not optimized for the chemical landscape. GAUCHE defines such kernels for strings, fingerprints, and graphs that operate on a range of widely used molecular representations [Griffiths and et al., 2022]. In this study, we used the Tanimoto kernel, which is defined for binary vectors $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$ for $d \geq 1$ as:

$$k_{\text{Tanimoto}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \frac{\langle \mathbf{x}, \mathbf{x}' \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - \langle \mathbf{x}, \mathbf{x}' \rangle},$$

where $\|\cdot\|$ represents the Euclidean norm [Griffiths and et al., 2022].

One ML method that has had particular success in molecular property prediction are Gaussian Processes (GPs) [Obrezanova et al., 2007, Griffiths and et al., 2022], a non-parametric class of models that use a kernel to produce a Gaussian predictive distribution over every data point [Obrezanova et al., 2007]. One of the major limitations of using GPs in practice is computational scaling; inference has $O(N^3)$ time complexity and $O(N^2)$ memory, where N is the number of training

points [Rasmussen and Williams, 2005]. This can be mitigated with sparse GP approximations, but these introduce approximation error and can be more difficult to tune [Quiñonero-Candela and Rasmussen, 2005]. Tree-based ensembles are typically more efficient for larger data sizes [Lakshminarayanan et al., 2017]. We implemented GP models using the `Gauche` framework [Griffiths and et al., 2022], which provides kernels optimized for molecular input [Griffiths and et al., 2022]. For fingerprint-based representations, we used the Tanimoto kernel [Ralaivola et al., 2005, Moss and Griffiths, 2020, Griffiths and et al., 2022], which operates on binary vectors. For PDV, we used a radial basis function (RBF). Preliminary tests indicated that for GPs using the Tanimoto kernel where applicable offered significant performance benefits; however for SVM, the RBF kernel was sufficient for all types of data.

2.3.4 Bayesian neural networks

Although NN are a popular choice in QSAR research, though they are often outperformed by more traditional methods [Baskin et al., 2008, Koutsoukas et al., 2017]. Decision trees and related ensemble models tend to perform stronger on smaller datasets, overfit less, and don't rely on a static 3D molecular structure like NNs do [Baskin et al., 2008, Koutsoukas et al., 2017]. NNs are often deterministic; however, they can be transformed into probabilistic models through several approaches. BNNs introduce priors on the weights and compute a posterior distribution on those weights given the input data, providing uncertainty quantification. This can be done by approximations, including Monte Carlo dropout, variational inference, or ensembles [Gal and Ghahramani, 2016]. To improve computational efficiency, several strategies exist for converting NNs to BNNs. One straightforward yet computationally expensive approach is to replace all linear layers with Bayesian layers (full Bayesian transformation, or full-BNN). Replacing only the final layer is computationally lighter (last-layer transformation, or last-layer-BNN). One particularly efficient approach is Variational Bayesian Last Layers, which use a variational formulation that can be trained with only quadratic complexity in the last layer width [Harrison et al., 2024] (variational transformation, or var-BNN). Bayesian approaches also act as forms of regularization [Burden and Winkler, 2009], as dropouts help to prevent overfitting.

While standard feed-forward neural networks (NNs) are deterministic, they can be transformed into probabilistic models. We tested two architectures: (i) NN-α

with two hidden layers of sizes [128, 64] and dropout after each hidden layer and (ii) NN-β with two hidden layers of size [128, 128] and dropout applied before the output layer. Each model was trained using the Adam optimizer with early stopping on validation loss. We used Bayesian transformations to convert NNs into Bayesian neural networks (BNNs), and implemented them using `PyTorch` [Fey and Lenssen, 2019] with ReLU activations and dropout regularization ($p = 0.2$).BNNs introduce priors on the weights and compute a posterior distribution given the data, providing uncertainty quantification. This can be done by approximations such as Monte Carlo dropout, variational inference, or ensembles [Gal and Ghahramani, 2016]. One straightforward yet computationally expensive approach is to replace all linear layers with Bayesian layers (full-BNN), using Gaussian priors $\mathcal{N}(0, 0.1^2)$ on all weights. The second approach we implemented was Variational Bayesian Last Layers (VBLL), which maintains a mean-field variational posterior $q(\mathbf{W}) = \mathcal{N}(\boldsymbol{\mu}_W, \mathrm{diag}(\boldsymbol{\sigma}_W^2))$ over the last-layer weights and is trained by maximizing the evidence lower bound, which can be thought of as minimizing the reconstruction loss plus KL divergence $D_{\mathrm{KL}}(q(\mathbf{W})\|p(\mathbf{W}))$ from a standard normal prior, scaled by $1/N$ [Harrison et al., 2024] (VBLL transformation). The VBLL also learns a scalar observation noise variance. All BNN variants estimated predictive distributions with 100 Monte Carlo forward passes at inference.

Unlike standard deterministic ML methods, Bayesian methods model distributions over predictions using Bayes' rule, allowing for the quantification of uncertainty within parameters and labels. Bayesian methods treat the model parameters as random variables with distributions. Uncertainty can also be estimated from other non-deterministic methods, including ensemble [Lakshminarayanan et al., 2017], bootstrap modeling [Palmer et al., 2022], and federated learning [Hanser, 2023].

We decomposed the uncertainty values derived from the probabilistic methods into epistemic and aleatoric components when applicable. Epistemic uncertainty is model-driven and can be reduced by collecting either more or higher quality training data. Aleatoric is data-driven and reflects the intrinsic noise in the labels. For the GP and VBLL models, we derive epistemic uncertainty from the posterior variance or weight sampling and aleatoric uncertainty from a learned observation noise term [Rasmussen and Williams, 2005, Harrison et al., 2024]. We did not decompose the uncertainty from other models such as BNN variants, in which the variance across Monte Carlo weight samples primarily reflects epistemic uncertainty [Kendall and Gal, 2017, Gal and Ghahramani, 2016]; and NGBoost and QRF, in which there is no

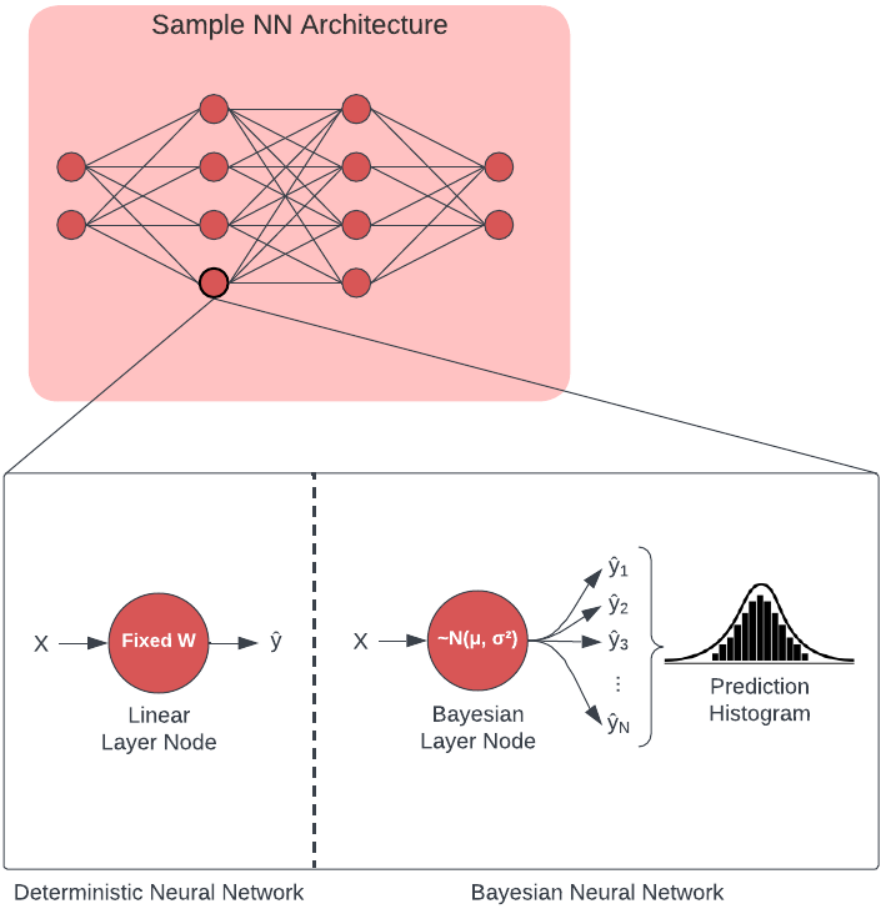


Figure 2.2: Deterministic versus Bayesian neural network architectures. Deterministic networks use fixed weights W to produce point predictions, while Bayesian networks treat weights as distributions $\mathcal{N}(\mu, \sigma^2)$, generating prediction histograms through multiple stochastic forward passes to quantify epistemic uncertainty.

straightforward mechanism to separate the aleatoric and epistemic components of uncertainty from learned distributional parameters or quantiles [Duan et al., 2020, Meinshausen, 2006].

2.4 Molecular similarity and activity cliffs

Distance-based reasoning rely on the assumption that the data embeddings contain quality information. When the labels are corrupted or the important features are scarce, similarity scores over-rely on compounding false or useless information, and the assumption that similar molecules have similar properties breaks down. Bajusz et al. [2015] et al. found that while the Tanimoto coefficient can be effective with binary fingerprints, changing the similarity formula yields minimal improvement if the underlying representation is weak [Bajusz et al., 2015]. This problem is further complicated by activity cliffs (ACs), which are defined as structurally similar molecules with large property differences, often due to a subtle chemical change [Dablander et al., 2023]. These molecules, which potentially contain the most important information for model prediction, can be confused with noise. Another barrier to distance-based reasoning is persistent noise across labels. The task of trying to determine if a molecule's value matches that of its neighbors becomes a lot harder when all of the neighbors have some form of noise as well.

2.5 Noise in molecular data

Real-world cheminformatics datasets contain label noise arising from multiple sources across the drug discovery pipeline. Although noise is often modeled as being random, or drawn from a Gaussian distribution, other varieties of noise must be considered when working with experimental data. To properly model experimental noise, we must first understand where the noise is coming from.

Assay-related variability represents a primary source of label noise in bioactivity data. High-throughput screening assays are often laden with errors from physical phenomena such as edge effects, where compounds in wells exhibit different activities at plate boundaries due to evaporation or temperature gradients [Macarron et al., 2011]. Other examples include compound carryover between wells, incomplete liquid transfers, and air bubbles [Inglese et al., 2006]. Detection reagents or unstable compounds can degrade over time, which leads to drift within a single plate run.

This turns into heteroscedastic noise correlated with plate position or read time [Brideau et al., 2003].

Instrument calibration and detection limits can impact measurement accuracy. Spectrophotometric assays near detection limits exhibit higher relative error, such that signal-to-noise ratios degrade for weak binders [Zhang et al., 1999]. Fluorescence quenching by test compounds introduces a systematic bia in fluorescence-based assays, resulting in false negatives for potent inhibitors that accidentally get absorbed at emission wavelengths [Simeonov et al., 2008]. Radiometric assays require background subtraction and decay corrections that accumulate error, particularly for low-activity compounds where the signal barely exceeds background [Zhang et al., 1999].

Another source of noise is biological variability. Even minor variations in cell culture conditions such as seeding density, plating efficiency, growth rate, and cell cycle distribution have the potential to affect downstream tasks such as cell metabolism and drug responsiveness. Even serum properties may introduce variability [Hatzis et al., 2014]. The speed of cell culture passages is even found to be a significant factor in behavior [Postnikova et al., 2018].

Data aggregation across multiple sources compounds these individual error sources. Common databases like ChEMBL, BindingDB, and PubChem aggregate measurements from hundreds of laboratories using a diverse set of protocols, compound purities, and quality control standards [Gaulton et al., 2017, Gilson et al., 2016]. The K_i or IC_{50} values from the same compound-target pair with between 10 and 20 reported values, often end up spanning two orders of magnitude apart, making it challenging to identify the underlying ground truth [Kramer et al., 2012]. All the sources of error mentioned above regarding inconsistent experimental protocols cannot be distinguished easily from the genuine underlying value without extensive, and often unrealistic amounts of metadata [Fourches et al., 2016].

Federated datasets, a method of data aggregation centered around privacy protection, presents a particularly challenging situation for noise. In pharmaceutical research and beyond, companies often want to merge collective information without exposing proprietary data to competitors. One example of such platforms is Lhasa Limited's federated learning platform Effiris. Teacher models are first trained separately on confidental data, and then used to label a shared public chemical data set. Those results are consolidated into a federated student model [Bassani et al., 2023]. While this is an effective approach of protecting intellectual property, it introduces

additional label noise through the prediction errors of all the teacher models. There also exists aleatoric uncertainty from the underlying experimental measurements, many of which likely come from differing protocols, resulting in compounding label noise.

Beyond the experimental setups, annotation errors in the data itself can arise from mis-registered structures, incorrect stereochemistry assignments, and salt form ambiguity [Williams et al., 2017]. Public databases are estimated to contain 1-5% structure errors from incorrect SMILES parsing, tautomer misassignment, or manual transcription mistakes during data entry [Young et al., 2008]. Ambiguous protonation states and counterions create representation-level noise where chemically distinct samples are classified as the same [Bento et al., 2020].

These noise sources can interact with each other and present in more ways than a simple Gaussian. Unlike artificial noise injection, real-world noise is neither independent, identically distributed, nor symmetric [Heid et al., 2023]. I wanted to address this complexity in my research by examining a variety of ways of both modeling and adding noise to data sets.

Although the general assumption in many of the noise analysis and mitigation literature is that noise, specifically label noise, will have a negative effect on the model, this is not necessarily the case. There are multiple instances of using noise as a method of regularization. Although not the optimal approach to counteracting overfitting in linear regression, adding artificial noise to labels is considered a form of regularization [Ullah and Welsh, 2024]. Furthermore, adding artificial noise drawn from a Gaussian distribution to the activation layers of a neural network is considered another form of regularization [Camuto et al., 2021]. These studies indicate that the presence of noise may act as a form of regularization in some contexts. It is important to address this potential of regularization from label noise in QSAR and QSPR models.

2.5.1 Artificial noise injection

Our objective is to evaluate the robustness of QSAR models against label noise. To do so, we "inject" artificial noise into the labels during training. Validation and test data remain free of noise. This artificial noise simulates real-world experimental noise. The clean target label y_i is replaced by a noisy label $\tilde{y}_i = y_i + \epsilon_i$, where ϵ_i is the artificial noise injected at the index i, the value of which is determined by a

specified noise strategy.

Experimental noise is often modeled as homoscedastic Gaussian noise and is added evenly across all labels. However, not all experimental noise is random. It may be heteroscedastic, where the variance depends on the molecule itself or experimental factors, or systematic, where biases from factors like assay conditions are introduced. We used a variety of noise strategies to model the different types of possible experimental noise that could arise in assay data coming from wet labs.

Noise Type	**Per-point scale s_i**	**Simulated real-world source**
Gaussian	σ	Random measurement error
Outlier	$\begin{cases} 3\sigma & \lvert z_i\rvert > 2 \\ 0.1\sigma & \lvert z_i\rvert \leq 2 \end{cases}$	Transcription errors, batch effects
Quantile	$\begin{cases} 2\sigma & y_i < Q_{10} \text{ or } y_i > Q_{90} \\ 0.1\sigma & Q_{10} \leq y_i \leq Q_{90} \end{cases}$	Systematic error in tail domains
Threshold	$\begin{cases} 2\sigma & \lvert y_i\rvert > 1 \\ 0.1\sigma & \lvert y_i\rvert \leq 1 \end{cases}$	Regime-dependent assay errors
Valprop	$\sigma\left(1 + 0.1\lvert y_i\rvert\right)$	Percentage-based measurement uncertainty
Hetero	$\sigma\sqrt{0.1 + 0.05\lvert y_i\rvert}$	Heteroscedastic measurement precision

Table 2.1: Regression noise injection strategies. For each point i, noise $\varepsilon_i \sim \mathcal{N}(0, s_i^2)$ is added to the target y_i, where the scale s_i is defined per row. Here $z_i = (y_i - \bar{y})/\sigma_y$ is the standardised target and Q_{10}, Q_{90} are the 10th and 90th percentiles of the target distribution.

Although classification tasks were not explicitly tested in this study, NoiseInjects implements six strategies that mirror the regression set: uniform, class-imbalance, binary-asymmetric, instance, class-dependent, and confusion-directed flipping. This is conducted with stochastic label flipping with flip probability p, such that p replaces σ as the noise scaling factor.

The parameter σ serves as a noise scaling factor across all strategies. For Gaussian noise, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ is applied uniformly. For outlier noise, samples identified

as outliers (z-score > 2.0) receive noise from $\mathcal{N}(0, (3\sigma)^2)$ while normal samples receive $\mathcal{N}(0, (0.1\sigma)^2)$. For quantile noise, samples above the 90th or below the 10th percentile of the target distribution receive noise scaled by $2.0\times$, while central samples receive $0.1\times$. For threshold noise, samples with $|y| > 1.0$ (on normalized data) receive the higher $2.0\times$ multiplier, while those within receive $0.1\times$. For value-proportional, $\epsilon_i \sim \mathcal{N}(0, (\sigma(1 + 0.1|y_i|))^2)$, using a multiplicative formulation where noise scales proportionally with the absolute target value. And finally, for heteroscedastic, noise variance is computed as $\alpha\sigma^2 + \beta\sigma^2|y_i|$ with $\alpha = 0.1$ and $\beta = 0.05$. All other multipliers remained constant, meaning that while the relative noise distribution differs between strategies, the difficulty scaling controlled by σ is consistent. The detailed progression of label corruption across noise levels is seen in Figure 2.3.

2.6 Learning with noisy labels

Numerous studies have explored learning with noisy labels (LNL) in the broader context of ML and specifically within QSAR models. Song et al. [2023] provides a comprehensive review of deep LNL, identifying five main approaches: modifying architectures to better handle noise in training, employing regularization to reduce overfitting to noise, adjusting loss functions to be less sensitive to outliers or label flips, post-processing to adjust the weights for noisy data, and sample selection to select presumably clean data to pre-train. Although some QSAR models can overcome noise, predictive performance depends on the algorithm. Cortes-Ciriano et al. [2015] examined 12 different algorithms across 12 datasets and 10 noise levels, finding that no single algorithm was overall best at noise; certain algorithms perform better or worse at different levels of noise. This is a recurring theme in LNL research: There is no universal solution to handling noise in cheminformatics data. This is consistent with the No Free Lunch Theorem, which states that there is no single learning algorithm that can outperform all others in all data distributions [Wolpert and Macready, 1997]. This suggests that there is no single best model or molecular representation for noise robustness; it depends on the underlying data. Certain techniques can be more effective with different types of noise. For example, small-loss selection works particularly well for symmetric noise [Song et al., 2023]. Some techniques that may be robust to experimental noise may not be fit for the particular QSAR task at hand. Combinations of LNL approaches have yielded promising results, specifically sample selection combined with semi-supervised learning on re-

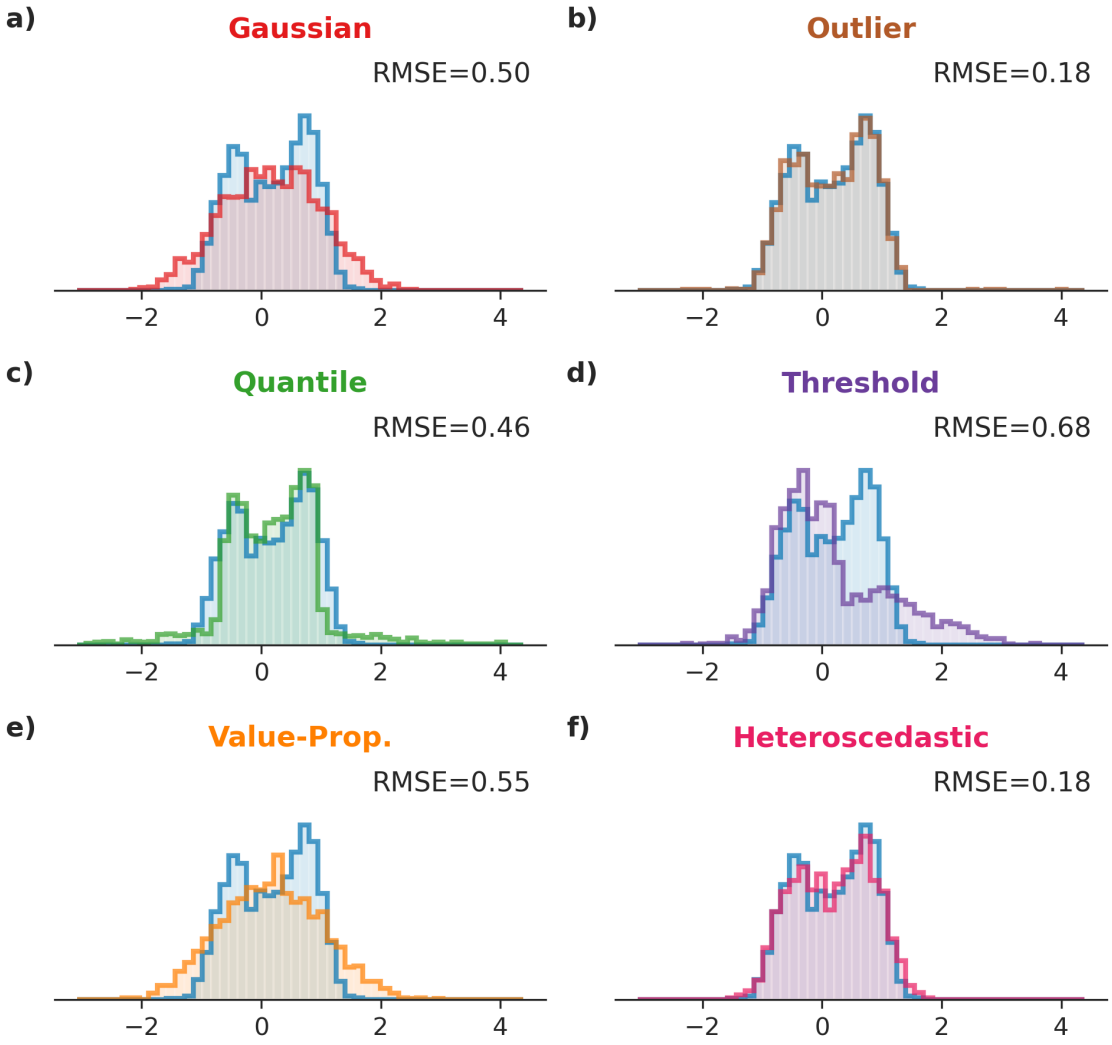


Figure 2.3: Effect of each noise injection strategy on the QM9 HOMO–LUMO gap label distribution (blue) at $\sigma = 0.5$. The RMSE annotated in each panel is computed between the clean and the noisy labels for that strategy (i.e. the magnitude of perturbation). a) Gaussian (red); b) Outlier (brown); c) Quantile (green); d) Threshold (purple); e) Value-Proportional (orange); and f) Heteroscedastic (pink).

jected data, presumably noisy samples. Many LNL techniques were developed for image classification tasks. The data properties of molecular datasets differ significantly, so adjustments are needed to adapt some of these techniques [Song et al., 2023].

2.7 NoiseInject: a benchmarking framework, and the precompute pipeline

In an extension of this work, we developed an open-source benchmarking framework called NoiseInject to evaluate model robustness and uncertainty calibration under artificial label noise for both regression and classification tasks. The framework operates on label arrays (NumPy arrays or pandas Series) and applies artificial noise to the training labels while preserving the integrity of the test set.

The framework computes standard regression metrics (RMSE, R^2, mean absolute error [MAE]) and classification metrics (accuracy, precision, recall, and F1 score, both macro- and weighted-averaged, with per-class breakdowns). For probabilistic models, it additionally computes uncertainty-calibration metrics: expected calibration error (ECE), empirical coverage at 1σ and 2σ, mean prediction-interval width, and the Spearman correlations between predicted uncertainty and both absolute error and injected noise. It reports noise robustness metrics, including the noise-performance degradation slope and retention percentage, to quantify performance degradation across noise levels. Results are returned as structured `pandas` DataFrames (per-noise-level values and an aggregate summary) for downstream analysis.

The evaluation pipeline is model-agnostic: it operates on the predictions of any externally trained estimator, including scikit-learn-compatible models and `PyTorch` networks. Optional reference wrappers that turn a point estimator into one producing per-sample uncertainty are also provided: a Gaussian process (via Gauche), split-conformal prediction for scikit-learn-compatible models, and Monte Carlo dropout for `PyTorch` networks. Worked examples demonstrating integration with common QSAR workflows (molecular descriptors, fingerprints, and GNN embeddings) are provided in the repository.

The software is available under an MIT license [Punt, 2026] and is designed to integrate into existing cheminformatics or other data pipelines, allowing for systematic

benchmarking of QSAR noise robustness.

To enable high-throughput experimentation across molecular representations, model architectures, and noise conditions, we developed a custom Python-Rust pipeline optimized for computational efficiency and experimental flexibility. The architecture divides computational responsibilities by language strengths: Rust handles preprocessing (SMILES tokenization, fingerprint generation, noise injection, normalization, binary serialization), while Python manages model training, hyperparameter optimization, and statistical analysis.

The framework supports both quantum-mechanical datasets (QM9 [Ramakrishnan et al., 2014b] via PyTorch Geometric) and Polaris Hub endpoints [Wognum et al., 2024]). Multiple splitting strategies are implemented: random splits for baseline comparisons, scaffold splits via DeepChem [Ramsundar and et al., 2019] to prevent structural leakage, and domain-based clustering using seven methods (random, fingerprint k-means, descriptor-based, Butina, Splito, scaffold, molecular weight).

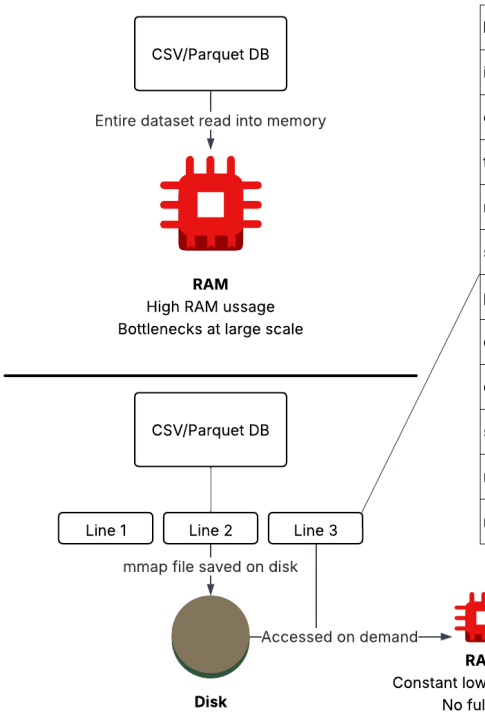
We support the following representations: ECFP4 (encoded with `RDKit` [Landrum] using radius $r = 2$ and length $d = 2048$), SNS [Dablander et al., 2024], PDV, physicochemical descriptor vectors (PDV: 200+ RDKit descriptors), one-hot-encoded SMILES (canonical and randomized), mol2vec [Jaeger et al., 2018a], ChemBERTa [Chithrananda et al., 2020], and MHG-GNN embeddings [Kishimoto et al., 2023]. All representations are precomputed during Rust preprocessing and stored in memory-mapped files for zero-copy access.

Each bit within the ECFP4 fingerprint encodes the hashed identity of a circular substructure. One downside of these hashed fingerprints are collisions, when chemically distinct substructures collide in the same index, resulting in representation-level noise. To address this issue, we also used a collision-free alternative for pooling ECFP substructures, *Sort & Slice* variant [Dablander et al., 2024]. Substructures are sorted by prevalence in training, and then the top L are sliced to form a binary vector. This process avoids hashing entirely.

Traditional approaches load entire datasets into memory from CSV or Parquet files, creating RAM bottlenecks at scale and redundant deserialization overhead across experimental replicates. Our pipeline instead serializes preprocessed data into custom memory-mapped binary files (`.mmap`) that enable on-demand access with constant low memory footprint regardless of dataset size (Figure 2.4).

Each mmap file stores a sequence of fixed-format entries separated by newline delimiters. Entry structure includes: length-prefixed UTF-8 SMILES strings (iso-

meric, canonical, optionally randomized), the raw target value (float32, 4 bytes), optional molecular representations (SNS: uint8[16] = 16 bytes, ECFP4: uint8[32] = 32 bytes, bit-packed SMILES one-hot encodings with length prefix), the processed target value incorporating noise and normalization (float32, 4 bytes), and optional domain label (uint8, 1 byte). Canonical SMILES are cached in SQLite (`smiles_db.sqlite`) to avoid redundant RDKit standardization across replicates. This format reduces storage by 60–80% compared to text-based serialization while enabling zero-copy memory mapping—Python and Rust access the same disk-backed memory pages without deserialization, maintaining constant RAM usage regardless of dataset size.



Field Name	Required?	Data Type & Size	Size
isomeric_smiles	Yes	UTF-8	Variable length
canonical_smiles	Yes	UTF-8	Variable length
target_value	Yes	float32	4 bytes
randomised_smiles	Optional	UTF-8	Variable length
sns	Optional	uint8[16]	16 bytes
processed_target	Yes	float32	4 bytes
domain_label	Optional	uint8	1 byte
ecfp4	Optional	uint8[32]	32 bytes
smiles_ohe	Optional	bit-packed uint8[]	Variable size
randomised_smiles_ohe	Optional	bit-packed uint8[]	Variable size
newline separator	Yes	ASCII '\n' character	1 byte

Figure 2.4: Memory-mapped file architecture contrasted with traditional in-memory loading. Top: CSV/Parquet approaches load entire datasets into RAM, creating bottlenecks at scale. Bottom: Memory-mapped binary files enable on-demand line-by-line access with constant low RAM usage. Right: Entry-level structure showing byte-aligned fields with fixed-size representations (target values, fingerprints, domain labels) and length-prefixed variable-length fields (SMILES strings, one-hot encodings).

The framework implements a variety of model architectures spanning multiple modeling families include: tree ensembels (RF, QRF [Meinshausen, 2006], XGBoost, NGBoost [Duan et al., 2020]), Gaussian Processes (`Gauche` [Griffiths and et al., 2022] with Tanimoto/RBF/Matérn kernels, heteroscedastic GP [Kersting et al., 2007], evidential kernel variants), neural networks (DNN, MLP, residual MLP, factorization MLP, and RNN and GRU on SMILES), conformal prediction wrappers (split CP, heteroscedastic CP with uncertainty-weighted conformity scores) [Liu et al., 2020, Shafer and Vovk, 2008], and LNL methods (Meta Weight Net, DivideMix, confident learning, small-loss selection, MentorNet, curriculum learning, multi-stage cleaning) [Li et al., 2020, Northcutt et al., 2021b, Han et al., 2018, Zhao et al., 2025, Shu et al., 2019, Jiang et al., 2018]. Bayesian transformations (full-BNN, last-layer-BNN, variational-BNN [Harrison et al., 2024]) are available for all neural architectures to transform linear nodes within a NN architecture into Bayesian nodes, converting a probabilistic NN into a BNN.

Fifteen loss functions enable investigation of noise-robust objectives: standard regression losses (MSE, MAE, Huber, smooth L1), robust losses (Cauchy, log-cosh, focal, truncated), quantile losses ($\tau \in \{0.1, 0.5, 0.9\}$), heteroscedastic losses predicting mean and variance, evidential losses with Normal-Inverse-Gamma priors (standard, Cauchy, Laplace variants), adaptive Barron loss with learnable robustness parameter, and domain-aware losses (domain-weighted, domain-balanced, per-domain heteroscedastic, adaptive domain, mixture domain). Six molecular distance metrics support sample weighting and outlier detection: Tanimoto coefficient for fingerprints, Euclidean/cosine/Mahalanobis for continuous representations, maximum mean discrepancy (MMD), and optimal transport.

Optuna [Akiba et al., 2019] provides Bayesian hyperparameter tuning with support for conditional parameters, pruning, and multi-objective optimization. Models can be trained with default hyperparameters, tuned values, or user-provided JSON configurations. Uncertainty estimates are extracted from probabilistic models (GP posterior variance, QRF quantile spread, NGBoost distribution parameters, BNN weight distributions, conformal prediction intervals) and saved to disk when applicable. Per-epoch training and validation metrics are optionally logged for convergence analysis and early stopping diagnostics.

The split-language architectures was design to leverage the capabilities of both langauages. Python was left to hand dataset loading from standard repositories, molecular embedding requiring complex Python-only libraries, and model train-

ing. Rust handles the CPU-intensive pre-processing tasks and noise injection. This framework is optimised for rapid and low-storage modeling, relying on Rust's speed for pre-processing and zero-copy data access via (`.mmap`) files.

All representations were precomputed using a hybrid Rust-Python pipeline. Rust was used to perform various heavy-computation tasks involved in feature extraction and serialization, while Python was used for model training and evaluation.

2.8 Evaluation methodology

Root mean squared error (RMSE) and the coefficient of determination (R^2) were primarily used to compare QSAR predictive performance. σ is the noise scaling factor shared by all strategies (Table 2.1): each strategy uses σ to control the magnitude of injected noise, so that $\sigma = 0$ corresponds to clean labels and increasing σ produces progressively noisier labels, regardless of how the noise is distributed across samples.

Pairwise statistical comparisons between deterministic and probabilistic architecture counterparts (e.g., NN-α vs NN-α BNN variants, RF vs QRF) were conducted using the Wilcoxon signed-rank test, with $\alpha = 0.05$ (two-sided). To assess whether model rankings remain consistent across different noise injection strategies, we computed Kendall's coefficient of concordance (Kendall's W), which ranges from 0 (no agreement) to 1 (complete agreement). Values above 0.7 indicate strong agreement in rankings across strategies.

When evaluating uncertainty, we assessed the relationship between predicted uncertainty and prediction quality. Spearman's rank correlation coefficient (ρ) was used to quantify the relationship between predicted uncertainty (u_i) and absolute error ($|y_i - \hat{y}_i|$), as well as between predicted uncertainty and injected noise magnitude. Higher correlations indicate that the model's uncertainty estimates reliably identify predictions that are likely to be incorrect.

For probabilistic models, empirical coverage was evaluated at 1σ and 2σ confidence intervals by computing the proportion of test predictions where the true value falls within $\hat{y}_i \pm k\hat{u}_i$:

$$\text{Coverage}(k\sigma) = \frac{1}{N}\sum_{i=1}^{N} 1\big[|y_i - \hat{y}_i| \leq k\hat{u}_i\big],$$

such that $\hat{u}_i$ is the predicted uncertainty for sample i and $k \in \{1, 2\}$. Theoretical target coverages are defined as 68% at 1σ and 95% at 2σ for Gaussian distributions.

The Expected Calibration Error (ECE) was computed by binning predictions into deciles by predicted uncertainty and measuring the weighted average absolute difference between predicted uncertainty and observed error:

$$\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} \big|\bar{u}_b - \bar{e}_b\big|,$$

where B_b is the set of predictions in bin b, $\bar{u}_b$ is the mean predicted uncertainty in that bin, and $\bar{e}_b$ is the mean absolute error. Lower ECE indicates better-calibrated uncertainty estimates.

To evaluate the effect of label noise, we examined model performance degradation under increasing artificial noise with $\sigma \in \{0, 0.1, 0.2, \ldots, 1.0\}$ using the Noise Degradation Slope (NDS), calculated as the slope of R^2 versus noise level:

$$\text{NDS} = \frac{dR^2}{d\sigma}.$$

Values closer to zero indicate that noise has less of an effect on performance, negative values indicate a higher sensitivity to noise. Positive slopes, which were not observed in this study, would indicate that the noise *improves* model performance. Configurations with baseline $R^2 \leq 0.6$ were excluded from robustness analysis, as poor performance on clean labels tends to remain poor for noisy labels as well, producing misleadingly shallow or even positive slopes that represent poor performance across the board.

A two-way analysis of variance (ANOVA) decomposition was conducted separately for each noise strategy to identify the relative contributions of molecular representation and model architecture choice on both predictive performance and noise robustness. We chose ANOVA with η^2 effect sizes rather than pairwise significance tests because our primary question is variance attribution: how much of the variation in robustness is explained by model architecture versus molecular representation. Pairwise tests can establish whether two models differ significantly but cannot partition the total variance among factors—the key distinction our analysis requires. This per-strategy approach avoids inappropriate aggregation across fundamentally different noise types. For a given metric y (either R^2 at fixed noise or NDS):

$$y_{ijr} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijr},$$

where α_i represents the effect of model architecture i, β_j represents the effect of molecular representation j, $(\alpha\beta)_{ij}$ is the interaction term, and ϵ_{ijr} is the residual for replicate r.

The proportion of variance explained by each factor was calculated as the effect size η^2:

$$\eta^2_{\text{factor}} = \frac{SS_{\text{factor}}}{SS_{\text{total}}},$$

using Type I (sequential) sum of squares, where SS denotes the sum of squares. Both factors (model architecture and molecular representation) are treated as fixed effects, and the 10 experimental replicates per scenario provide the error term.

ANOVA relies on the assumption that observations both within and across groups are independence. Across the whole set of models and representations in this study, some pairs produced similar predictions that are not independent. This causes the residual degrees of freedom to increase, while the residual sum of squares barely moves. To handle this issue, we computed pairwise Spearman rank correlations between all models and between all representations across noise strategies. Models whose profiles correlated at $\rho > 0.99$ with another model were excluded, including quantile regression forests (QRF), which were very similar to RF, as well as different NN architectures. The GP model was also excluded because different kernels are used for different molecular representations. This was not a problem with SVM as the RBF was used across all representations.

Across representations, SNS correlated at $\rho > 0.90$ with the ECFP4 fingerprint and was excluded to avoid inflating representation degrees of freedom, as both encode overlapping circular substructure information. We also computed intraclass correlation coefficients ICC(1,1) for all model pairs. ICC(1,1) measures the proportion of total variance attributable to between-subject differences, with values near 1.0 indicating that two models rank configurations almost identically. The full redundancy and ICC tables are reported in Appendices B.2–B.4.

Retention percentage was also used to measure the proportion of baseline performance preserved in higher-noise conditions:

$$\text{Retention}(\%) = \frac{R^2(\sigma_{\max})}{R^2(\sigma_{\min})} \times 100$$

High retention values represent stronger noise robustness.

2.9 Conclusions

Write the Background chapter's closing paragraph.

Metric	Definition	Used to assess
RMSE	$\sqrt{\frac{1}{N}\sum_i (y_i - \hat{y}_i)^2}$	Predictive accuracy (lower is better)
R^2	$1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$	Variance explained by the model; comparable across datasets and noise levels
NDS	$dR^2/d\sigma$ (slope of R^2 vs σ)	Robustness to label noise; values near zero indicate stability, more negative values indicate sensitivity
Wilcoxon signed-rank	Paired non-parametric test, $\alpha = 0.05$ (two-sided)	Pairwise comparison of probabilistic vs deterministic counterparts (e.g. NN-α vs BNN-α)
Kendall's W	Coefficient of concordance, $[0, 1]$	Stability of model rankings across noise strategies; $W > 0.7$ indicates strong agreement
ANOVA η^2	$SS_{\text{factor}}/SS_{\text{total}}$ (Type I)	Variance attribution to model architecture vs molecular representation
ICC(1,1)	Intraclass correlation, $[0, 1]$	Profile-level redundancy between models prior to ANOVA
Spearman's ρ	Rank correlation, $[-1, 1]$	Calibration of predicted uncertainty against (i) absolute error and (ii) injected noise magnitude
Coverage	$\frac{1}{N}\sum_i \mathbb{1}[\lvert y_i - \hat{y}_i\rvert \leq k\hat{u}_i]$	Fraction of test points whose true value lies within $\hat{y} \pm k\hat{u}$; targets 68% at $k = 1$, 95% at $k = 2$
ECE	$\sum_b \frac{\lvert B_b\rvert}{N} \lvert\bar{u}_b - \bar{e}_b\rvert$	Calibration of predicted uncertainty: deviation between binned predicted uncertainty and mean error (lower is better)

Table 2.2: Summary of metrics used in this study, their definitions, and their intended interpretation. $\hat{y}_i$ and $\hat{u}_i$ denote the predicted value and predicted uncertainty for sample i; σ is the noise scaling factor (Table 2.1).

Chapter 3

Noise Robustness Across Molecular Representations and Model Architectures

Preamble. The work in this chapter and in Chapter 4 is adapted from Punt, A., Hanser, T., Werner, S., and Morris, G.M., in preparation for the *Journal of Cheminformatics*. Most of the figures, tables, and text in these two chapters are therefore either identical or similar to the content of that manuscript. A signed Statement of Authorship for this jointly authored work, covering both chapters, is included in the front matter.

3.1 Introduction

Numerous studies have explored Learning with Noisy Labels (LNL) in the broader context of ML, as well as specifically for QSAR models. Song et al. [2023] provide a comprehensive review of Deep LNL, identifying five main approaches: (i) modifying architectures to better handle noise in training; (ii) employing regularization to reduce overfitting to noise; (iii) adjusting loss functions to be less sensitive to outliers or label flips; (iv) post-processing to adjust the weights for noisy data; and (v) sample selection to select presumably clean data to pre-train. Cortes-Ciriano et al. [2015] examined 12 different algorithms across 12 datasets and 10 noise levels, finding that no single algorithm was overall best at maintaining predictive performance in the presence of noise; certain algorithms perform better or worse at different levels of noise. This is a recurring theme in LNL research: there is no universal solution to

handling noise with experimental endpoints. This is consistent with the 'No Free Lunch Theorem', which states that, *there is no single learning algorithm that can outperform all others in all data distributions* [Wolpert and Macready, 1997].

Although numerous studies have explored the effects of noise on ML algorithms for molecular property prediction and have used a handful of representations as inputs to these algorithms [Kolmar and Grulke, 2021], direct comparisons between different molecular representations and how they respond to label noise remain limited. The choice of molecular representation certainly affects the predictive performance of QSAR models. Deng et al. [2023] conducted a large-scale evaluation of 62,820 models and found that Extended Connectivity Fingerprint (ECFP) fingerprints frequently outperform the representations of learned graph neural networks. Additionally, experimental noise is often modeled as Gaussian-distributed — the argument being that according to the Central Limit Theorem, aggregated experimental measurements from multiple independent sources of error exhibit a Gaussian distribution [Kolmar and Grulke, 2021]. However, Heid et al. [2023] used mean-variance estimation and bias-variance decomposition to show that noise can be tied to specific modalities, for example structure dependence.

3.2 Datasets and noise injection

The majority of experiments were conducted on the QM9 molecular property dataset [Ramakrishnan et al., 2014b], which contains approximately 130,000 small organic molecules with pre-computed quantum-mechanical properties obtained from density functional theory (DFT) calculations at the B3LYP/6-31G(2df,p) level of theory [Ramakrishnan et al., 2014b]. Each experiment was replicated 10 times with a distinct random seed and a randomized subset of $N = 10{,}000$ molecules. Preliminary experiments were done with $N = 5{,}000$, 10,000, 20,000, and 30,000, resulting in similar performance at all sizes except $N = 5{,}000$ which experience minor degradation. Experiments that involved tracking uncertainty values were only run once. For each experimental replicate, a new scaffold-based train/validation/test split (80/10/10) implemented by DeepChem [Ramsundar and et al., 2019] was generated from this fixed subset. Scaffold splits were used to minimize structural overlap between sets. While predictive performance typically drops when switching from random splitting to a more challenging scenario like scaffold splitting, the use of scaffold splitting discourages the QSAR model from overfitting [Heid et al., 2023].

We selected HOMO–LUMO energy gap as the primary prediction target in QM9, as it captures electronic characteristics relevant to molecular reactivity and stability [Islam et al., 2019, Hüllermeier and Waegeman, 2021]. The HOMO–LUMO gap is a well-established target for quantum chemistry benchmarks, and is defined as the difference between the highest occupied and lowest unoccupied molecular orbital energies, directly relating to a molecule's electronic excitability, charge transfer capability, and chemical stability [Fediai et al., 2023]. We also assumed that thanks to the consistent calculation method, and notwithstanding approximations in the level of theory used, the QM data were "free" of noise.

To assess whether noise robustness results generalize beyond the HOMO–LUMO gap, we evaluated model configurations across three additional molecular property datasets with experimentally measured endpoints. From the OpenADMET initiative [Open ADMET Consortium, 2026], we used LogD (lipophilicity, N = 5,039) and Caco-2 efflux permeability (N = 2,161). We also selected hERG Ki data from ChEMBL [Zdrazil et al., 2023], following a protocol inspired by Landrum and Riniker [2024]: filter for binding assays and deduplicate by median pChEMBL value. We then removed compounds with inter-assay standard deviation > 1.0 log unit [Zdrazil et al., 2023]. This results in N = 1,482 compounds. All three datasets were evaluated using all six noise injection strategies and eleven levels of noise as used in the QM9 experiments. 5-fold scaffold cross-validation was conducted using GroupKFold on Murcko scaffolds. Models used the hyperparameters listed in Appendix B.1, with representation-specific SVM kernels detailed in Appendix B.12. For binary vector-based representations, a Tanimoto kernel was used for SVM, while an RBF kernel was used for continuous-valued vectors.

Prior to modeling, all molecular structures were sanitized and canonicalized using `RDKit` to remove invalid valence states and standardize atom and bond typing [Landrum]. Target values were mean-centered and normalized to zero mean and unit variance. Data loading and pre-processing were performed with a global random seed to ensure reproducibility.

3.3 Molecular representations

In this study, we evaluated a variety of molecular representations spanning path-based fingerprints, circular fingerprints, physicochemical descriptors, string encodings, and pretrained embeddings. We used two binary fingerprints: an ECFP4

fingerprint and the *Sort & Slice* (SNS) fingerprint [Dablander et al., 2024]. The ECFP4 fingerprint uses circular substructures with radius $r = 2$, hashing them into a standard $d = 2048$-bit vector using `RDKit` [Landrum]. Each bit encodes the presence of one or more substructures capturing the local chemical environment around each atom. One downside of hashed fingerprints is bit collisions, in which chemically distinct substructures map to the same index, resulting effectively in representation-level noise. To address this issue, the SNS fingerprint uses a collision-free alternative based on the same Morgan circular substructures with radius $r = 2$ [Dablander et al., 2024]. Substructures are sorted by prevalence in the training set, and the top L ($L = 1024$) are sliced to create a binary vector, providing the substructure-based circular fingerprint in our comparison. We also used 200-dimensional physicochemical descriptor vectors (PDVs) computed from RDKit molecular descriptors (`MolecularDescriptorCalculator`), encompassing molecular weight, LogP, topological polar surface area, topological connectivity indices, partial charges, VSA bins, EState indices, hydrogen bond donor/acceptor counts, and functional group counts, among others [Cherkasov et al., 2014]. These PDV features are z-score normalized such that the mean and standard deviation are computed on the training set, with constant-variance features set to unit scale. We also used one-hot-encoded (OHE) SMILES strings [Bertoni et al., 2020], generated with `RDKit` [Landrum] as well as mol2vec embeddings [Jaeger et al., 2018b], which apply a Word2Vec-inspired approach to molecular substructures. Finally, as a representative GNN-based learned representation, we included graph embeddings generated by Molecular Hypergraph Grammar GNNs (MHG-GNN) [Kishimoto et al., 2023]. MHG-GNNs are a GIN-based autoencoder pre-trained on 1.34 million PubChem molecules using β-VAE loss, producing 1024-dimensional embeddings through iterative message passing. MHG-GNN was originally developed for material science, and has demonstrated strong performance on property prediction tasks for polymers, photoresistors, and chromophores [Kishimoto et al., 2023].

3.4 Representation governs clean performance, architecture governs robustness

Considering the two main components of a QSAR model, the molecular representation and the model architecture, we sought to understand how each contributes

to predictive performance and noise robustness. ANOVA decomposition across six noise strategies on models to predict HOMO–LUMO gap from QM9 demonstrates that for predictive performance (R^2 at $\sigma = 0.3$), the model–representation interaction term is the largest source of variance across all six strategies, followed by model architecture and representation (Table 3.1; Figure 3.1a). However, for noise robustness as measured by the noise degradation slope (NDS), this ordering shifts. The choice of model architecture is instead the largest source of variance, while the interaction term drops to second and representation third. This observed inversion makes sense because the molecular representation dictates the chemical information the model receives, while the architecture determines how efficiently that information is used. The interaction term reflects the idea that these two are not independent. In the context of label noise robustness, representation has less of an impact since the features are not corrupted by noise. Meanwhile, the model's regularization, ensembling, and uncertainty handling have a stronger impact on how it handles noise.

The impact varied by type of noise, represented by noise strategy (Figure 3.1b). The strategies which degraded in performance the most, threshold and value-proportional, both corrupted label regions, forcing models to rely on their inductive biases. This translates to the model architecture explaining a significant portion of the variance. On the other hand, outlier noise only affects statistical outliers. For outlier noise, model and interaction effects were both smaller.

The interaction between model architecture and representation is seen in Figure 3.2. While some model architectures, like SVM and full BNNs maintain consistent noise robustness scores across all representations, others like NN-β display greater variation. Model robustness on one representation strongly predicts robustness on another (Spearman $\rho = 0.73$, $p = 0.007$ for ECFP4 vs PDV; 11 models). However, the scatter around the diagonal reflects the interaction term identified by the ANOVA.

We filtered out model–representation configurations with baseline predictive performance $R^2 < 0.6$, excluding 66 configurations where poor clean-data performance would produce misleadingly shallow degradation slopes (Appendix B.5). We evaluated all model architectures under increasing label noise on the PDV representation across six strategies, considering both predictive performance as measured by R^2 as well as the noise degradation slope (NDS). We observe that NDS varies substantially across architectures and noise strategies, as seen in Figure 3.3.

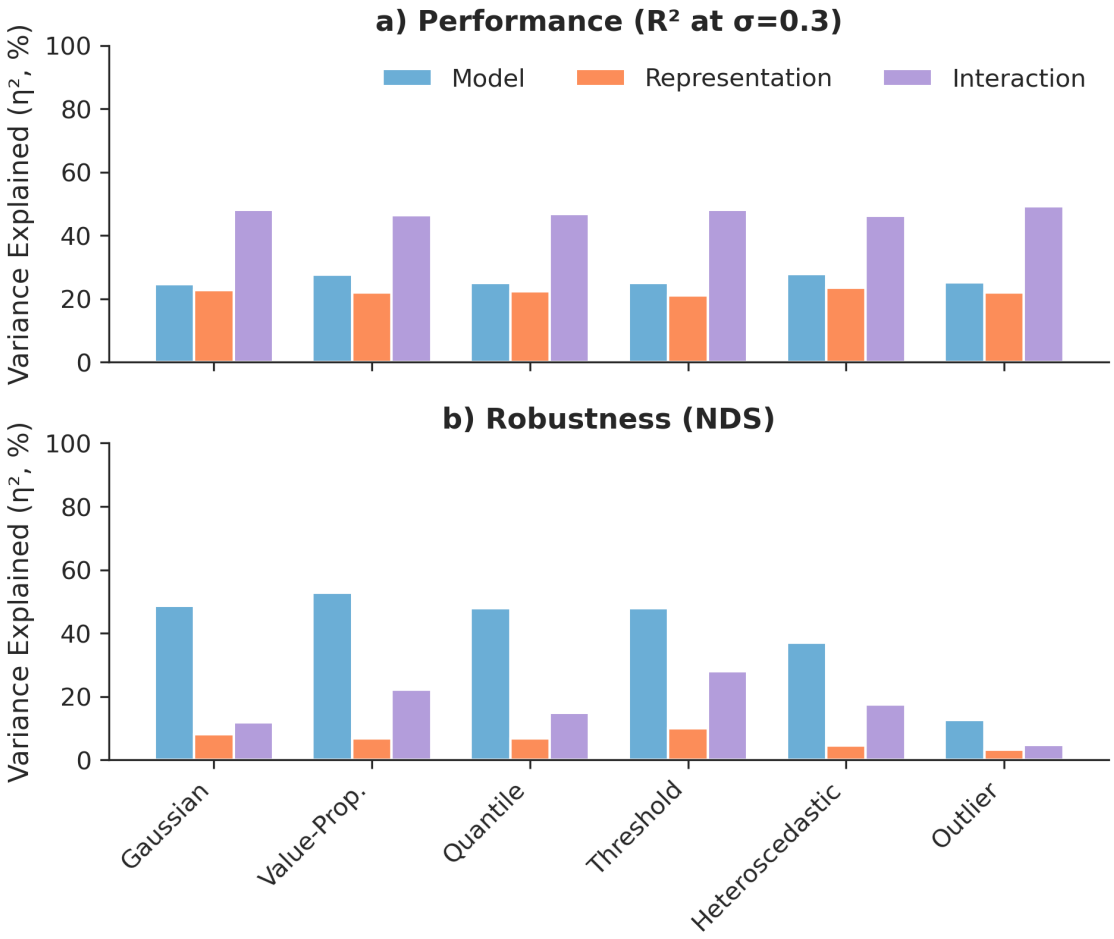


Figure 3.1: ANOVA variance decomposition (η^2, %) for a) predictive performance (R^2 at $\sigma = 0.3$) and b) noise robustness, quantified by the noise degradation slope (NDS), by noise strategy. Lower (more negative) NDS indicates faster performance loss under noise.

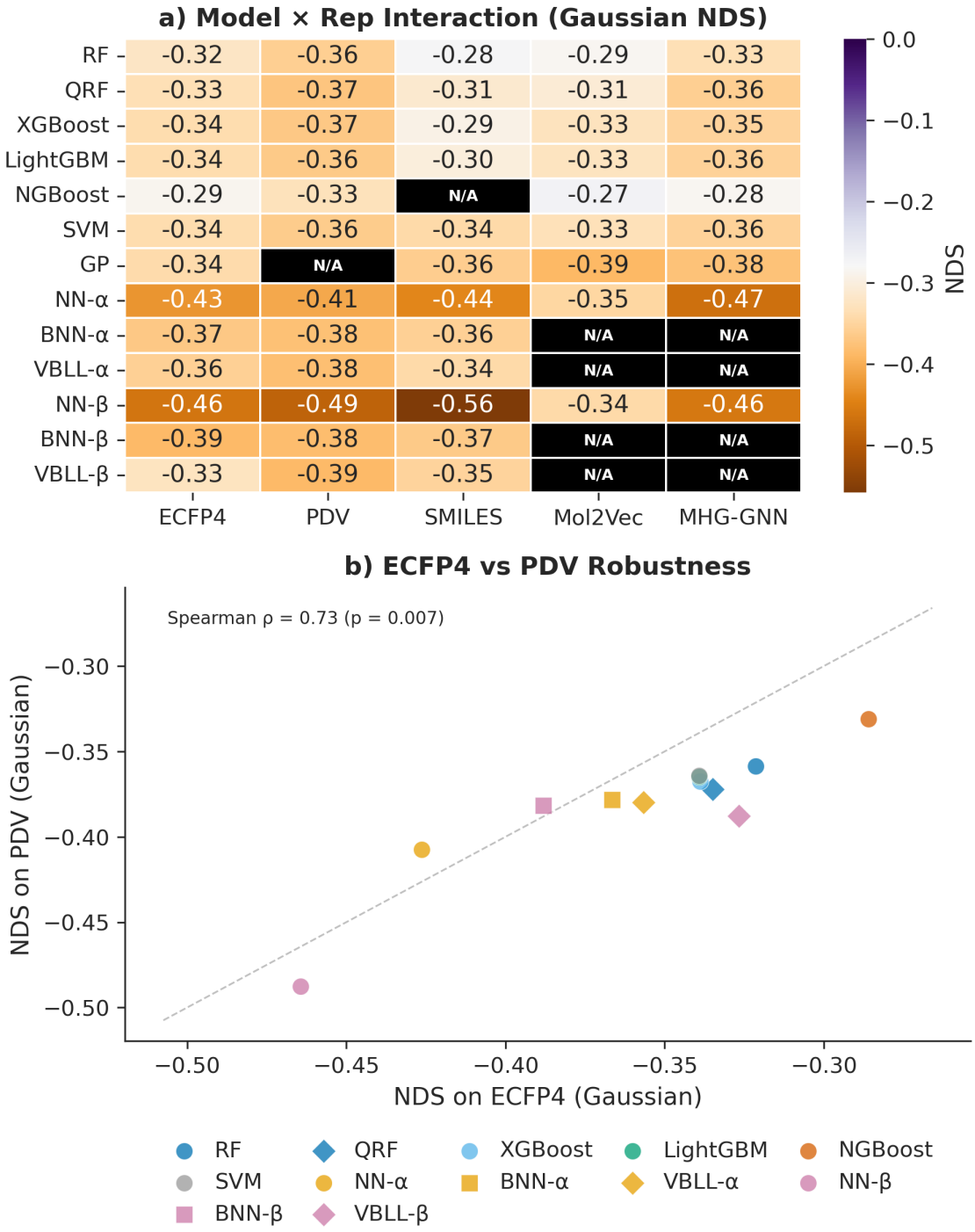


Figure 3.2: Representation–model interaction effects on the QM9 HOMO–LUMO gap. a) Noise degradation slope (NDS) heatmap showing model robustness across representations under Gaussian noise; lower (more negative) NDS indicates faster degradation. "N/A" indicates configurations excluded due to baseline $R^2 < 0.6$ or kernel incompatibility (GP uses a Tanimoto kernel defined on binary vectors, which is incompatible with PDV). b) NDS on PDV versus ECFP4 fingerprint for each model under Gaussian noise, with Spearman correlation.

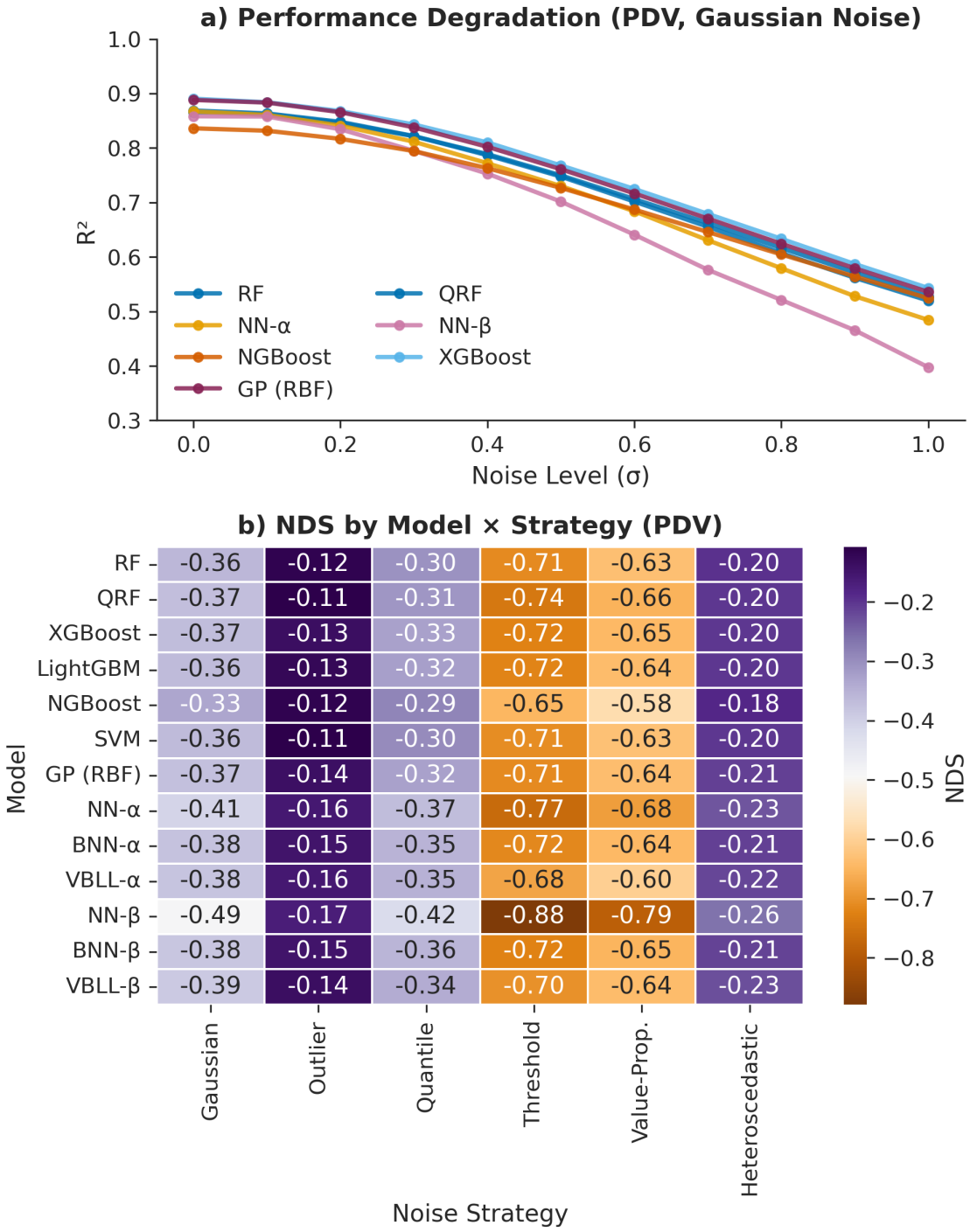


Figure 3.3: Global overview of noise robustness on the QM9 HOMO–LUMO gap. a) Performance degradation curves (PDV, Gaussian noise) showing R^2 versus noise level σ for representative models. b) Noise degradation slope (NDS) heatmap across all model architectures and noise strategies on PDV; lower (more negative) NDS indicates faster degradation under noise. The Tanimoto GP is absent because its kernel is incompatible with PDV (see Section 2); a separate GP with an RBF kernel is included for PDV-specific analyses.

Table 3.1: ANOVA variance decomposition by noise strategy on the QM9 HOMO–LUMO gap. Each cell reports η^2 (%), the share of variance explained by that factor. Columns: **Model** = model architecture effect; **Rep** = molecular representation effect; **Inter.** = model×representation interaction; **Resid.** = residual unexplained variance. "Performance" uses R^2 at $\sigma = 0.3$ as the outcome; "Robustness" uses the noise degradation slope (NDS). Bold entries highlight the dominant factor in each row.

	Performance (η^2, %)				**Robustness (η^2, %)**			
Strategy	**Model**	**Rep**	**Inter.**	**Resid.**	**Model**	**Rep**	**Inter.**	**Resid.**
Gaussian	24.6	22.8	**48.1**	4.5	**48.7**	8.2	11.9	31.2
Quantile	25.1	22.4	**46.9**	5.6	**47.9**	6.8	14.9	30.4
Threshold	25.0	21.2	**48.0**	5.8	**48.0**	10.1	28.0	13.9
Heteroscedastic	27.9	23.5	**46.2**	2.5	**37.0**	4.5	17.5	41.0
Value-prop.	27.8	22.0	**46.5**	3.8	**52.8**	6.9	22.2	18.2
Outlier	25.3	22.0	**49.2**	3.5	12.7	3.2	4.7	**79.3**

3.5 Probabilistic models are no more noise-robust than deterministic ones

To further explore the impact of model architecture on noise robustness, we examined variants of the same core architecture. Specifically, Bayesian transformations were used to explore the impact of the Bayesian framework on noise. By comparing models with the same core architecture, we observe that with full Bayesian transformations, representation explains less than 11% of NDS variance ($p > 0.25$) while variational Bayesian inference on the same NNs is representation-dependent. On MHG-GNN and mol2vec representations, VBLL models were unstable, such that all 24 VBLL × {MHG-GNN, mol2vec} configurations were excluded due to baseline $R^2 < 0.6$. Further work could involve more thoroughly-tuning these models and exploring different core architectures to adapt to these embedding-style inputs.

In general, the full Bayesian transformations reduced the degradation in performance with Gaussian noise the most, as seen in Figure 3.4. Both full BNN and VBLL transformations significantly improved robustness for both NN-α and NN-β (Table 3.2). Full transformations are more computationally expensive, making VBLL a potentially stronger approach to handling noise with NNs when working

with larger data sets or modeling architectures. The choice of transformation varies by representation, as seen in Appendix B.8, with SMILES receiving the largest benefits from Bayesian transformations. QRF was significantly less robust than RF (Table 3.2), suggesting that quantile regression adds complexity and flexibility which may result in overfitting.

Table 3.2: Wilcoxon signed-rank tests for Bayesian and probabilistic transformations on the QM9 HOMO–LUMO gap. **Base** is the deterministic reference model; **Variant** is the probabilistic/Bayesian transformation tested against it. **Δ NDS** = mean change in noise degradation slope (NDS) from base to variant; positive values mean the variant is more robust (slope closer to zero). **Sig.** marks pairs that are statistically significant at $p < 0.05$ (asterisk).

Base	**Variant**	**Δ NDS**	p**-value**	**Sig.**
NN-α	BNN-α	+0.056	2.9×10^{-10}	*
NN-α	VBLL-α	+0.061	1.2×10^{-6}	*
NN-β	BNN-β	+0.096	2.9×10^{-11}	*
NN-β	VBLL-β	**+0.124**	1.2×10^{-7}	*
RF	QRF	**−0.022**	1.6×10^{-10}	*

3.6 Relative model robustness is largely strategy-independent

NGBoost and SVM showed the smallest degradation in predictive performance across all strategies, while NN-β showed the steepest (Table 3.3). When sorting models by NDS, their ranks across noise strategies were consistent: Kendall's $W = 0.92$ ($p = 2.7 \times 10^{-8}$; 11 models, 6 strategies), indicating that a model's relative robustness is largely strategy-independent. The same ranking pattern holds on the ECFP4 fingerprint (Appendix B.6). Robustness is also largely decoupled from baseline predictive performance: across models on PDV under Gaussian noise, NDS clusters near -0.38 regardless of baseline R^2 (Appendix B.7), so a high-performing model is not automatically a more robust one.

The strategies differed not only in predictive performance severity, but also in their ability to discriminate between architectures. The spread of NDS across models

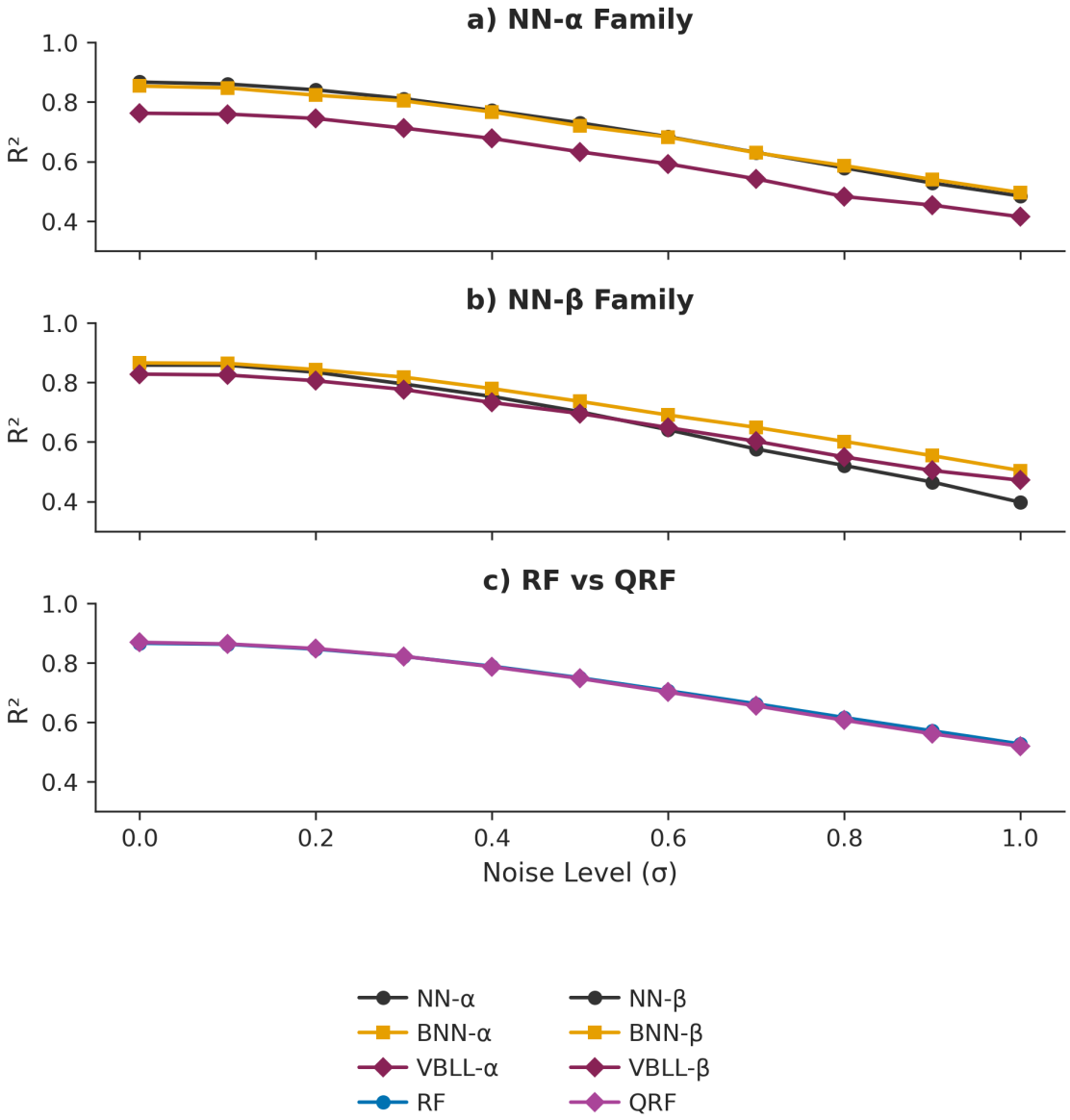


Figure 3.4: Neural network family comparison on the QM9 HOMO–LUMO gap with N=10k: R^2 versus σ under Gaussian noise on the physicochemical descriptor vector (PDV) representation for a) NN-α family, b) NN-β family, and c) RF versus QRF. Heteroscedastic results (not shown) are qualitatively consistent.

Table 3.3: Noise degradation slope (NDS) by model on PDV (QM9 HOMO–LUMO gap), ranked by mean across six strategies. Lower |NDS| (slope closer to zero) indicates greater robustness. Strategy abbreviations: **Gauss.** = Gaussian, **Quant.** = Quantile, **Thresh.** = Threshold, **Hetero.** = Heteroscedastic, **Val.-P.** = Value-Proportional. Bold rows mark the two most robust models; bold cell entries mark each strategy's most robust configuration.

Model	**Gauss.**	**Outlier**	**Quant.**	**Thresh.**	**Hetero.**	**Val.-P.**	**Mean**
NGBoost	**−0.33**	−0.12	**−0.29**	**−0.65**	**−0.18**	**−0.58**	**−0.36**
SVM	−0.36	**−0.11**	−0.30	−0.71	−0.20	−0.63	−0.38
RF	−0.36	−0.12	−0.30	−0.71	−0.20	−0.63	−0.39
LightGBM	−0.36	−0.13	−0.32	−0.72	−0.20	−0.64	−0.40
VBLL-α	−0.38	−0.16	−0.35	−0.68	−0.22	−0.60	−0.40
XGBoost	−0.37	−0.13	−0.33	−0.72	−0.20	−0.65	−0.40
VBLL-β	−0.39	−0.14	−0.34	−0.70	−0.23	−0.64	−0.41
BNN-α	−0.38	−0.15	−0.35	−0.72	−0.21	−0.64	−0.41
BNN-β	−0.38	−0.15	−0.36	−0.72	−0.21	−0.65	−0.41
NN-α	−0.41	−0.16	−0.37	−0.77	−0.23	−0.68	−0.44
NN-β	−0.49	−0.17	−0.42	−0.88	−0.26	−0.79	−0.50

differed by noise strategy, with outlier noise having a spread of 0.06 and threshold noise having 0.23, presumably exposing architectural differences that are masked by other types of noise. As seen in Table 3.3, NGBoost was the most resistant to noise across strategies. With respect to the Variational Bayesian Last Layer (VBLL) models, VBLL-α showed a similar pattern, outperforming all tree-based models except NGBoost under threshold and value-proportional noise despite ranking below them under Gaussian noise. This suggests that its learned noise variance is most beneficial when label corruption is systematic rather than uniform. This observation with VBLL is not alone; predictive performance on clean data is not well-correlated with noise robustness. In fact, some of the models with the lowest NDS across strategies such as NGBoost and SVM did not perform particularly well on clean data. These models may be more prone to picking up on noise rather than underlying signals in the data.

As established by the ANOVA, model architecture and the model–representation interaction term dominate NDS variance. A handful of models stood out as being noise-robust regardless of representation, namely SVM and full BNNs. Both methods rely on inductive biases, SVM on margin maximization and BNN on weight priors. However, other models like RF and NN-β show the opposite pattern. These models can be robust to noise, but only when paired with particular representations. The pairings differ by model; however, PDV stood out as having particularly strong robustness to noise. This suggests that PDVs are preferable when dealing with noisy data.

While representation may not be a dominant determinant in NDS variance, the model–representation interaction term does have an impact. For SMILES, model choice explains over 91% of robustness variance, while on PDV, model choice explains 72%. Across all six noise strategies, PDV configurations produced the shallowest mean NDSs across representations, with its best performance coming with models with strong inductive biases like SVM and full BNN. It appears that PDV's compact feature space complements margin-based and Bayesian regularization. Embeddings struggled, particularly when paired with NNs. These patterns are aligned with the importance of the model–representation interaction term, indicating that representation choice modulates noise robustness within a given architecture.

Taking a closer look at model choice for noise robustness, we ranked all model–representation configurations on QM9 by their Gaussian NDS. Tree-based ensembles dominate the top 10: NGBoost appears five times, XGBoost and RF twice each, and

LightGBM once. No neural-network configuration appears the top 10. Rankings are consistent across strategies and the top QM9 configurations maintain their advantage on ADME datasets, though SVM configurations rank higher on these ADME datasets than NGBoost.

3.7 The pattern holds across prediction targets

In addition to working with "clean" quantum molecular properties, we tested our findings on experimentally-obtained data, replicating our process of adding artificial noise. Unlike QM9, where $\sigma = 0$ corresponds to a noise-free baseline, the experimental endpoints carry their own measurement noise in both training and test labels; the injected σ therefore stacks on top of an unknown noise floor. Per-dataset NDS heatmaps across all model architectures and noise strategies are shown in Figure 3.5. The ANOVA on these additional datasets confirms that model architecture dominates robustness variance on all three datasets, as seen in Appendix B.10, and that the trends with respect to noise robustness generalize. NGBoost ranks first on both QM9 and external datasets, SVM also transfer quite well. Some models' predictive capabilities are limited on these external datasets, likely due to a combination of smaller training sets, narrower chemical coverage, and the unknown experimental noise in the labels. XGBoost and BNN variants suffer the most here, as seen in Figure 3.6a.

QRF was consistently less robust than RF on every external data set (Appendix B.11). As seen with QM9, choice of molecular representation has a minimal effect on noise robustness across all three external datasets, reinforcing the finding that model architecture, not representation, is the main driver of noise robustness.

3.8 Conclusions

These results lead to two main conclusions. The first is that noise robustness, defined as the rate at which predictive performance degrades with increasing label noise, is primarily determined by the model's training mechanism. The margin maximization of SVM, the weight priors of BNNs, and the ensembling in tree methods all appear to keep those models from fitting to noise. The choice of representation has a stronger influence on predictive performance than noise robustness, accounting for less than $\tilde{1}0\%$ of the variance in robustness. The interaction term between representation and

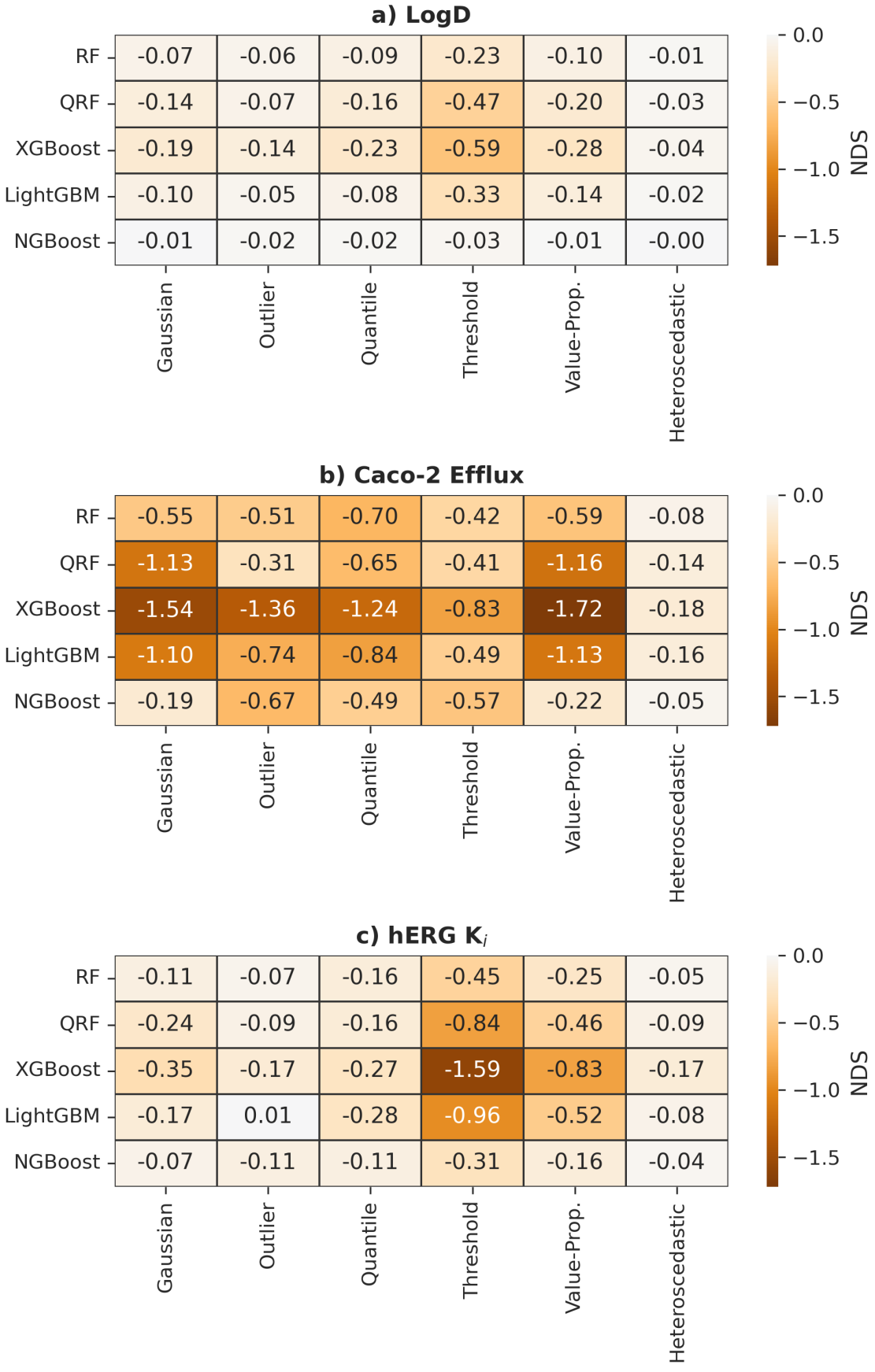


Figure 3.5: Noise degradation slope (NDS) heatmaps for three external validation datasets: a) LogD, b) Caco-2 Efflux, and c) hERG K_i. Lower (more negative) NDS indicates faster degradation under noise. Black cells indicate baseline $R^2 < 0.3$ for that (model, strategy) pair; "N/A" indicates filtered extreme values ($|NDS| > 2$).

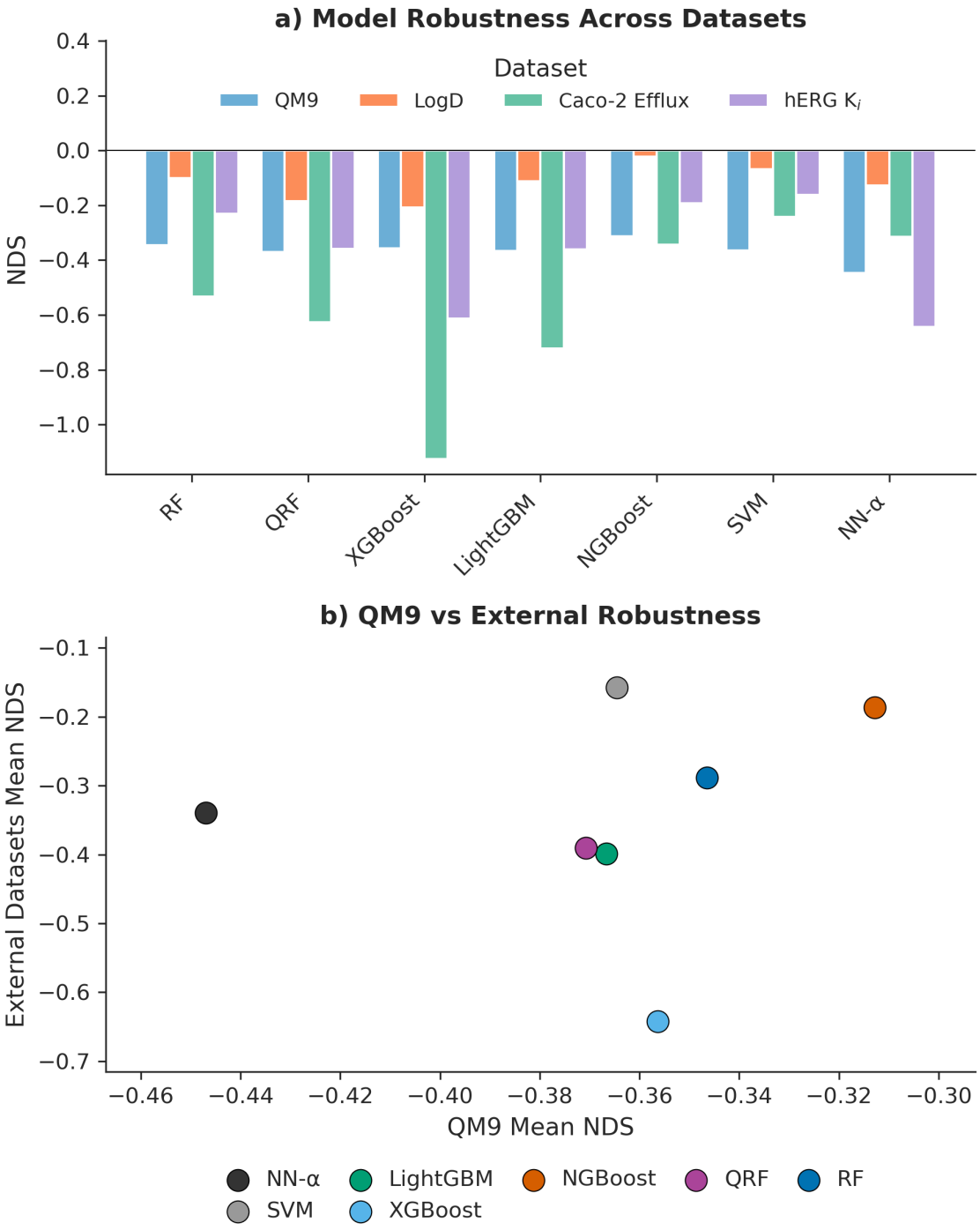


Figure 3.6: Comparison of noise robustness on QM9 (HOMO–LUMO gap, N = 10,000) and three external validation datasets: LogD (N = 5,039), Caco-2 Efflux (N = 2,161), and hERG K$_i$ (N = 1,482). a) Model robustness measured by the noise degradation slope (NDS) across all four datasets; lower (more negative) NDS indicates faster degradation under noise. b) QM9 vs external robustness: models with inherent robustness mechanisms (NGBoost, SVM) maintain their relative standing, while ensemble-dependent models (XGBoost) degrade on the external datasets.

model has a stronger impact; some models like SVM and full BNNs remain robust across all representations, while RF and NNs are robust only with certain ones. While the pattern of noise has a strong impact on predictive performance, it does not influence noise robustness rankings between models or representations much.

We investigated how a wide variety of molecular representations and model architectures perform under a regime of increasing label noise in QSAR modeling. We used ANOVA variance decomposition across six noise injection strategies on the QM9 HOMO–LUMO gap task to show that with regards to predictive performance, the interaction term between model and representation accounts for the most variance, with representation and model architecture contributing comparatively less. For predictive performance degradation under label noise, we look at the noise degradation slope (NDS), defined as the slope of R^2 with respect to the noise scaling factor σ, and see that model architecture is the dominant factor, while representation explains less than 10% of variance for most types of label noise.

Threshold and value-proportional noise produce the widest spread of NDS across models; outlier noise barely separates them. Model rankings are highly concordant across strategies (Kendall's $W = 0.92$), suggesting that a model's robustness to noise is independent of type of noise. Predictive performance on clean data is not correlated with noise robustness; NGBoost and SVM, the most noise-robust models, did not perform particularly well on clean data. SVM and full BNNs maintained consistent NDS across all representations, while embeddings (MHG-GNN, mol2vec) struggled with regards to noise robustness, especially with neural network architectures. On the flip side, QRF was significantly less noise-robust than RF across datasets, suggesting that quantile regression may be overfitting to noisy labels rather than absorbing them.

Chapter 4

Uncertainty Quantification Under Label Noise

Preamble. This chapter continues the study of Chapter 3 and is adapted from the same manuscript, Punt, A., Hanser, T., Werner, S., and Morris, G.M., in preparation for the *Journal of Cheminformatics*.

4.1 Introduction

Write Sec 4.1. Hirschfeld, Scalia and Norinder are cited in no source – add to refs.bib.

4.2 Models with an explicit noise term best track prediction error under noise

Kolmar and Grulke [2021] found that the mean predicted uncertainty derived from the GPs increases with the amount of label noise in the training data. We go beyond the population level and instead ask if a model's per-sample uncertainty tracks label noise. One key sign that a model is able to handle noise is its ability to track it, represented by the per-sample Spearman correlation between predicted uncertainty and noise magnitude (Table 4.1). While GPs and NGBoost gave the strongest correlations, BNNs were moderate. Learned embeddings (MHG-GNN and mol2vec) struggled, so much so that their correlations dropped to near zero or below on every model tested.

Table 4.1: Strongest and weakest model–representation combinations for uncertainty–noise correlation on the QM9 HOMO–LUMO gap (Gaussian noise strategy). Columns: **Unc-Noise** ρ = Spearman correlation between predicted uncertainty and injected noise magnitude $|\epsilon|$ (higher = better noise detection); **Unc-Error** ρ = Spearman correlation between predicted uncertainty and absolute prediction error; **ECE** = expected calibration error (lower = better-calibrated); **Cov.** 1σ / 2σ = empirical coverage at the 1σ (68% target) and 2σ (95% target) intervals.

Model	**Rep.**	**Unc-Noise** ρ	**Unc-Error** ρ	**ECE**	**Cov.** 1σ	**Cov.** 2σ
GP	**SNS**	**0.56**	0.23	0.14	71.2%	94.6%
GP	Morgan	0.53	0.22	0.14	71.4%	95.4%
NGBoost	**SNS**	**0.47**	0.16	0.13	69.3%	95.4%
NGBoost	**PDV**	**0.47**	**0.29**	**0.11**	69.0%	95.2%
NGBoost	PDV (binary)	0.44	0.25	0.13	69.8%	95.4%
NGBoost	Morgan	0.44	0.20	0.14	69.8%	95.3%
BNN-α	SNS	0.42	0.21	0.17	73.4%	95.1%
BNN-β	SNS	0.40	0.21	0.18	73.5%	95.1%
BNN-β	Morgan	0.38	0.20	0.18	72.1%	95.0%
BNN-β	PDV	0.38	0.24	0.19	72.1%	94.0%
Contrast: lowest correlations (graph/embedding representations)						
BNN-β	mol2vec	-0.05	0.02	1.33	50.6%	76.4%
VBLL-α	MHG-GNN	-0.05	0.09	13.41	26.6%	38.6%
VBLL-β	MHG-GNN	-0.03	0.10	15.34	31.1%	47.1%
VBLL-β	mol2vec	-0.03	0.15	23.86	44.8%	58.0%

4.3 NGBoost and the GP stay near target coverage while Bayesian variants miscalibrate

Samples containing label noise tend to produce large residuals in training, and every model has its own way of handling them. NGBoost contains predicted scales that increase to absorb residual [Duan et al., 2020]. The GP contains has a single global observation noise term. It does not absorb individual residuals, so its per-sample uncertainty derives primarily from the posterior variance rather than label noise [Rasmussen and Williams, 2005, Obrezanova et al., 2007]. BNNs treat these large residuals differently; they broaden their weight posteriors, increasing predicted uncertainty without explicitly modeling observation noise [Gal and Ghahramani, 2016, Kendall and Gal, 2017]. VBLL adds a learned noise variance to the BNN loss [Harrison et al., 2024], but this value is a global scalar, independent of the input. As seen in Table 4.1 and Appendix B.9, both BNNs and VBLLs achieve moderate correlations which improve when paired with fingerprint or descriptor-based representations. Among models that learn separate distributional parameters during training, GP and NGBoost produced the strongest correlations between uncertainty and label noise, though QRF, which separates parameters using quantile distributions, produced both poor label predictions and uncertainty-noise correlations. At the population level, mean predicted uncertainty increases with artificial noise. Although we expect to see aleatoric uncertainty increase with injected noise, for VBLL both the aleatoric and epistemic components increased (Figure 4.1b).

4.4 Uncertainty attribution is itself representation-dependent

MISSING RESULT: the GP uncertainty decomposition by representation.

4.5 Fingerprint representations carry the usable uncertainty signal

The choice of representation has a much stronger impact on per-sample uncertainty tracking than on noise robustness. Fingerprints and physicochemical descriptors

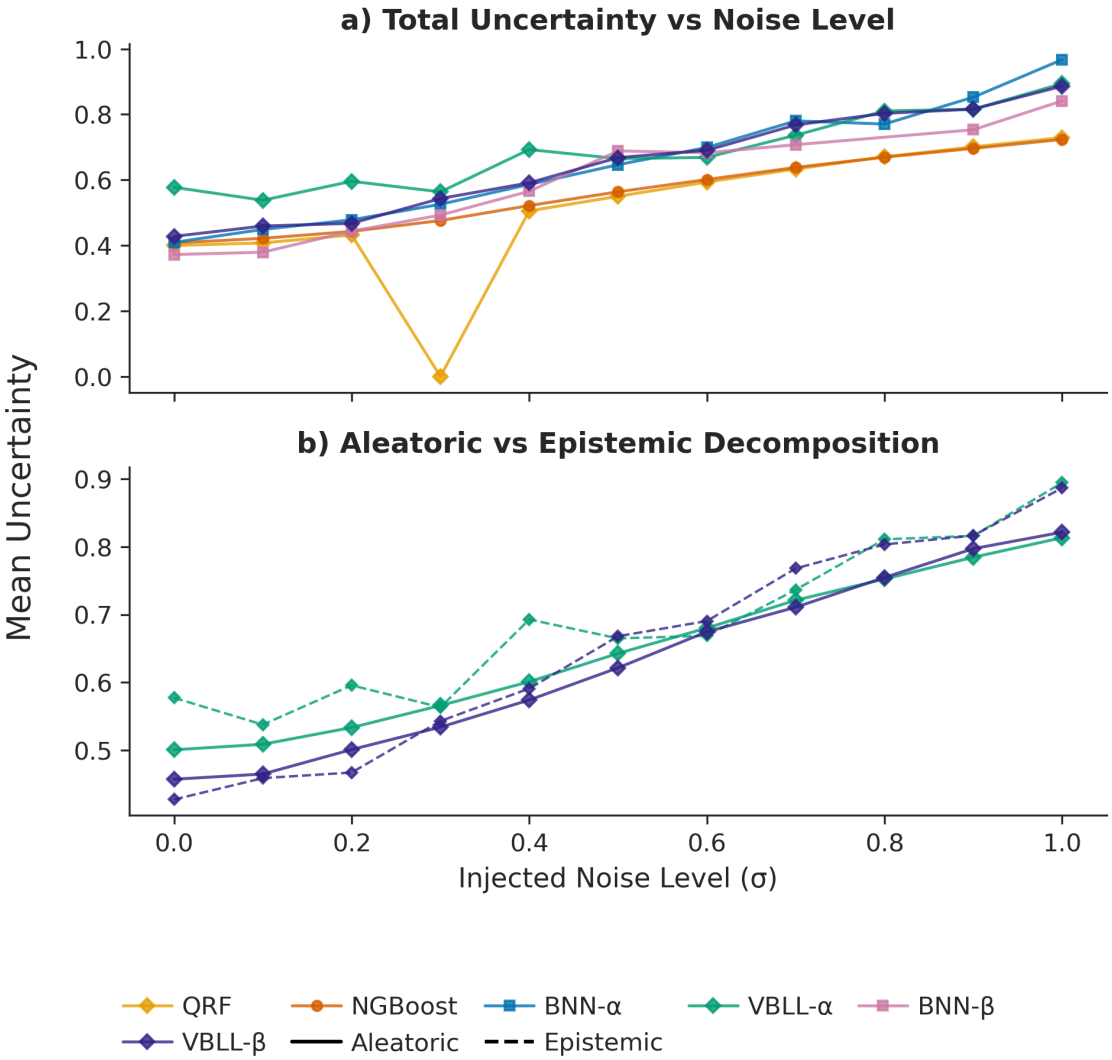


Figure 4.1: Uncertainty estimates under label noise on the QM9 HOMO–LUMO gap. a) Mean predicted uncertainty versus σ for probabilistic models (PDV, Gaussian strategy). b) Aleatoric and epistemic uncertainty components versus σ for VBLL models.

provided a clear signal with which models such as GP and NGBoost could separate noise-induced residuals from structural variation. However, learned embeddings such as MHG-GNN and mol2vec did not produce viable uncertainty-noise correlations (Table 4.1). Although these learned embeddings can be effective representations for many tasks, this research suggests that they provide challenges for per-sample noise tracking. However, there is room for further investigation, as only two embeddings were tested.

4.6 Conclusions

The second conclusion concerns whether a model's uncertainty estimates track per-sample label noise. Some models struggle, particularly those like BNNs whose uncertainty comes from a posterior over weights or QRF which does produce a full quantile, but is unable to successfully track noise. Models that fit a separate, per-sample scale or observation-noise parameter during training are able to track that label noise more effectively. NGBoost identifies noisy labels through a per-sample predicted scale that absorbs large residuals. Similarly, the GP absorbs these larger residuals with a per-sample posterior variance. Among representations, uncertainty tracking collapses with the learned embeddings MHG-GNN and mol2vec, while fingerprints and descriptors produce a viable signal. Neither strong predictive performance nor noise robustness implies noise tracking; again it is up to the model's architecture and how it handles loss.

Although the choice of representation carries less weight than that of models, we observed an interesting phenomenon. PDVs produce the strongest predictive performance on clean data, but are the least noise-resistant. Embeddings, particularly mol2vec, are the most robust to noise yet perform the worst noise tracking. At the end of the day, the choice of both model and representation comes down to your problem, including the amount of noise in your data, your objectives, and your compute limits.

With regard to uncertainty estimation, models that learn a separate scale or noise parameter during training, particularly NGBoost and Gaussian Processes, produced the strongest per-sample correlation between estimated uncertainty and injected noise magnitude. Fingerprint-based representations produced moderate-to-strong uncertainty-noise correlations, while the learned embedding MHG-GNN produced near-zero or negative correlations for all models. QM9 served as a rel-

atively clean baseline; it is computationally-derived with negligible measurement noise. The validation datasets, LogD, Caco-2 [Open ADMET Consortium, 2026], and hERG Ki [Zdrazil et al., 2023], tested are smaller and cover a narrow range of the chemical and biological space, and have an unknown inherent amount of noise from experimentally-determined values, on top of the added artificial noise. These validation datasets supported the QM9 findings, with XGBoost the notable exception, degrading on the external datasets. Future work should extend this type of noise robustness benchmarking to classification tasks and larger experimental datasets with better-known uncertainty estimates.

When working with noisy experimental data in QSAR settings, it is important to keep in mind that noise robustness and noise-aware uncertainty estimation are primarily driven by the model's mechanism of training, and to a certain extent the choice of molecular representation, usually with respect to the model. While Bayesian transformations on neural networks improve noise robustness, they do not produce uncertainty estimates that correlate particularly strongly with noise. Models like NGBoost and GPs, which include a learned noise or scale parameter in the training objective, are the most robust to noise across both various types of noise and datasets and often produce uncertainty estimates which track per-sample label noise. Pairing these models with fingerprint-based representations provides additional benefits in restricting, and potentially detecting and mitigating label noise.

Chapter 5

A Method for Building Adaptive Hybrid Representations

5.1 Motivation: the failure of similarity-based noise handling

There are a wide variety of approaches to Learning with Noisy Labels (LNL), spanning sample selection, meta-learning, curriculum learning, and iterative refinement [Song et al., 2023]. Cheminformatics data presents additional challenges in the form of high-dimensional molecular representations and dealing with a wide variety of chemical domains with various similarities and relationships. In this chapter, we explore a set of 12 LNL methods in the context of QSAR modeling. We examine the impact of uncertainty quantification, molecular distance metrics, and alternative loss functions in the hopes of improving performance under label noise. We tested 221 total configurations across LNL methods, loss functions, distance metrics, and molecular representations. Notably, we found that the inclusion of distance metrics substantially hurts performance and that molecular representation choice explains even less variance under LNL methods. We also identified the following LNL techniques which showed promise on the tasks at hand: heteroscedastic GP, curriculum learning, and sample selection methods. The work was meant to connect the representation and uncertainty quantification insights found in Chapter 3 with more classical noise mitigation techniques as a means of determining broader research directions.

One of the most prominent outcomes was that distance-augmented methods per-

formed significantly worse than ones that did not incorporate any sense of distance or molecular similarity. An ANOVA test was run on the different adapted factors and shows that distance metric usage is the dominant factor in variance in R^2.. When ranking the performance of methods, all 20 worst-performing configurations with respect to noise robustness used distance metrics, not a single one achieved both high baseline ($R^2 > 0.6$) and high noise retention ($> 80\%$). This pattern was observed across standard distance types (Euclidean, Manhattan, Cosine, Mahalanobis) and a chemistry-aware metric (Tanimoto coefficient).

Building upon the results in Chapter 4 which identified the use of distance or similarity measures as a major area of weakness with respect to learning with noisy labels and noise mitigation, we decided to investigate alternative featurization techniques. There are multiple possible explanations behind the poor performance of distance metrics, first being that a singular molecular representation lacks sufficient information for distance calculations, with and possibly even without noise. These representations all encode different types of molecular information, PDVs encode global molecular properties, ECFPs contain substructure information, and MPNNs such as MHG-GNNs capture 3D structural features. These single sources of data may not be sufficient for robust similarity estimation. Additionally, some of these representations are sparse and contain a large number of features that are not relevant to molecules of a given domain.

The main issue we identified here is that when a large proportion of dimensions are uninformative, similarity measures become dominated by those uninformative features, causing exponential growth in the number of irrelevant pairwise interactions as dimensionality increases. This is what we sought to address by developing a representation with informative features and low dimensionality. We do so by creating a budget-constrained hybrid representation, which takes high-importance features from a variety of sources. These hybrid representations concatenate the top-ranked features by importance from a selection of base representations.

In addition to evaluating hybrid representation quality for distance-based reasoning, our investigation evolved to explore alternative noise handling strategies. Initial experiments with sample removal methods revealed limitations when applied to Gaussian noise affecting all samples. This motivated a pivot toward sample weighting and label smoothing approaches, which do not require accurate binary classification of noisy versus clean samples but instead leverage continuous noise magnitude estimates.

This research aims to address the following questions:

1. Do hybrid representations achieve higher noise robustness (retention percentage) than their base representations?

2. Do the selected hybrid features help stabilize distance-based noise mitigation?

3. Does feature importance weighting improve effectiveness of hybrid representation creation, model training, and noise mitigation tasks?

4. When sample removal is suboptimal, can sample weighting or label smoothing provide superior noise handling under Gaussian corruption?

5.2 The adaptive hybrid representation

Here, we present KIRBy (Knowledge Integration by Representation), a method that takes a collection of frozen feature matrices for a dataset and returns a customized hybrid feature set optimized for a specific prediction task. The inputs can be anything that gives a feature vector per sample: fingerprints, descriptor sets, foundation model embeddings, gene expression signatures, histopathology features. From these sources, KIRBy selects a compact feature subset, searching over budgets, allocation strategies, importance methods, and downstream models. The search runs once per dataset. A portable Blueprint stores the result; applying it to new data does not require repeating the search.

5.2.1 Review of existing approaches

Prior research in this field shows promise, with conjoint fingerprints achieving stronger performance on binding affinity prediction than alone across a range of models [Xie et al., 2020]. Augmenting structural-based fingerprints like ECFP4 with bio-activity based fingerprints has also been successful at improving both predictive performance and scaffold-hopping [Laufkötter et al., 2019]. A very similar fusion strategy, in which features are selected from six non-hashed fingerprints and are combined by feature importance score within individual models, consistently improves predictive accuracy over the original fingerprints [Turkina et al., 2025].

The majority of the work in this space deals with combining relatively standard fingerprints and linear representations like SMILES [Xie et al., 2020, Laufkötter

et al., 2019]. However, graph-based representations capture important structural information that is often lost in fingerprints. We explore using GNN embeddings within these hybrid, fusion methodologies. We further expand on the mechanisms used to compute feature importance, concatenate features, and use these hybrids within similarity quantification and noise mitigation tasks.

So far, the most common strategy for combining feature sets has been concatenation (early fusion), which requires only a single downstream model [**?**]. It works when feature sets are homogeneous. When they are not, concatenation degrades [**?**], and Li and Deng et al. called the practice "misleading" when the goal is to measure the contribution of individual representation types [Deng et al., 2023]. Late fusion trains a separate model per source and aggregates predictions [**?**], avoiding the dimensionality problem. But cross-source feature interactions are lost. To recover cross-source interactions requires intermediate fusion (MOGONET [**?**], cross-attention [**?**]), but at the cost of end-to-end training on all sources; adding a new source means retraining from scratch. No interpretable feature subsets emerge from multiple kernel learning [**?**], which operates entirely at the kernel level. There remains, as Steyaert et al. noted, no decisive evidence that any of these strategies is broadly superior [**?**].

In practice, most research groups default to one representation and ignore the rest. A cheminformatics lab reaches for ECFP fingerprints; a deep learning group uses MoLFormer or ChemBERTa; a structural biology group extracts ESM-2 embeddings. These may encode quite different aspects of the same underlying biology. But they are rarely compared, let alone combined. This leaves signal on the table. When groups do attempt combination, the most accessible strategy is concatenation. It works at modest scales. Concatenating just three standard sources, however (ECFP4 at 2,048 dimensions, ESM-2 at 1,280, Mordred descriptors at ∼1,600), gives a ∼5,000-dimensional input for a dataset that may contain only a few hundred labeled samples. At these sample-to-feature ratios, more features mean worse predictions [**??**]. Models sensitive to distance, such as kNN and SVM, are affected most severely [**?**].

The pace of new model development shows no sign of slowing, and extends well beyond small molecules into protein language models [**??**], encoders built for antibodies [**??**], histopathology foundation models [**?**], and gene expression. If a new protein language model appears next month, must we retrain from scratch to determine whether it adds value? Forward compatibility is therefore essential: any practical method must incorporate new sources without retraining or architectural

modification. Deep fusion approaches fail this requirement, since adding a new modality means retraining the entire network. But treat encodings as independent frozen feature sets, and the problem disappears: a new model's output joins the pool and is tested alongside existing sources with no changes to the procedure.

5.3 The method as a search

Treating each encoding as a frozen feature set, rather than learning a joint embedding end-to-end, makes the approach modular: any new encoder's output can be added to the pool without retraining, and the selection process itself is decoupled from how the features were produced. Given K frozen representation matrices $\mathbf{R}_1, \ldots, \mathbf{R}_K$ for the same n samples, a label vector $\mathbf{y}$, and optional group assignments for cross-validation, the search finds which features from which sources best predict for a specified metric and downstream model. The output is a hybrid feature matrix of b columns, where b is the feature budget, together with a Blueprint that records all parameters needed to reproduce the identical feature selection on new data. The search first scores each source independently, then scores feature importance on training data within each cross-validation fold, allocates the budget among sources according to baseline performance, and scores candidate hybrid configurations via cross-validation. We retain the best configuration for each combination of downstream model and budget. Every selected feature traces back to its source and importance rank: a complete provenance record.

Inputs from different sources differ in scale, distribution, and missingness patterns, and naively pooling them before splitting would leak validation-set statistics into the training procedure, a well-documented source of optimistic bias in feature selection studies [**??**]. All preprocessing therefore runs within each cross-validation fold. We impute missing values with per-feature medians computed on the training fold and drop columns that are entirely missing. Binary feature matrices (molecular fingerprints, for instance) skip standardization, since their $\{0, 1\}$ encoding already carries a consistent scale. For all other features, we standardize to zero mean and unit variance using training-fold statistics only.

5.3.1 Component representations

We combine a variety of component, or base representations to create these hybrids. The framework is generalized such that it can accept any vector-based input as a base representation. Here, we use PDV, ECFP4, and MHG-GNN. These are all representations that cover a different domain of knowledge; PDVs describe physiochemical properties, ECFP4s contain information on substructures, and MHG-GNNs contain structural information directly from 3D molecular graphs such as functional group arrangements, ring systems, and branching.

Prior to hybrid construction, optional quality filtering is applied such that features with $> 95\%$ zero values are removed, working under the assumption that they provide minimal discriminative signals across the given molecular population. This threshold is derived from standard practice in molecular descriptor pre-processing [Todeschini and Consonni, 2009].

5.3.2 Feature scoring and budget allocation

Not all features in a given source are equally informative for a given task, and, for reasons that are not entirely clear, different importance methods disagree about which features matter. Tree-based measures favor features with high split gain, while gradient-based attributions reflect sensitivity in a neural network's loss landscape. To avoid committing to a single notion of importance, we score features using three methods, one from each family: LightGBM gain (tree-based), TreeSHAP (explanation-based), and DeepLift (neural attribution). We selected these three from an initial pool of eight methods after redundancy analysis on datasets showed that members of the same family gave highly correlated rankings. How to divide the total feature budget among sources is the role of the allocation strategy. We tested 31 candidates and retained 13 that appeared in top-performing configurations on multiple datasets and model types. The 13 retained strategies fall into three families. Score-based strategies divide the budget in proportion to transformed baseline scores: restricting features to the top k sources, for example, or penalizing redundancy between correlated sources. A floor-based variant guarantees every source a minimum allocation, while a greedy forward approach iteratively adds features from whichever source most improves the internal evaluator, with early stopping. Once we have the allocation, we select the top features from each source by importance score and concatenate them to form the hybrid.

Some features are too low-dimensional or too specialized to participate in the budget allocation stage: MACCS fingerprints (167 features), for instance, or log-likelihood ratios (3 features). Excluding them from the allocation would lose potentially useful signal; including them would distort the budget distribution because they cannot contribute many features. We handle these as augmentation: after the base hybrid is constructed, we pool the augmentation features, rank them by random forest feature importance, and append them only if they improve the internal evaluator score.

Given a total feature budget B, we test two strategies for allocating features from the given base representations. The first method uses greedy forward selection, iteratively adding top-ranked features from the representation that provides the largest performance improvement on the validation set. If the performance plateaus, it stops early.

1. **Split:** Divide training data into allocation-train (80%) and validation (20%) subsets using scaffold splitting.
2. **Initialize:** Set current feature set to empty, remaining budget to B.
3. **Rank features:** For each representation $r \in \{base_1, base_2, \ldots base_m\}$, train RFs using the base representations individually, then rank features by mean decrease in impurity.
4. **Iterate:** While remaining budget $\geq$ step_size and patience not exhausted:
   (a) For each representation, try adding the top step_size candidates from its ordered list to the current feature set.
   (b) Train a RF using the tentative feature set.
   (c) Evaluate R^2 on the validation set.
   (d) Select the representation whose addition yields the highest validation R^2.
   (e) Permanently add step_size features from the winning representation; remove from its candidate list.
   (f) If validation R^2 does not improve for patience consecutive rounds, terminate early.
5. Re-compute importance scores for the features of the new hybrid on the full training data. For each representation, select the top n_r features according to

the allocation determined above, where n_r is the number of features allocated to representation r during iteration.

This greedy approach works better on larger datasets, as the size of the validation set may not be enough to gain meaningful insights from.

The second method is performance-weighted allocation, where the distribution of features from each base representation is proportional to that representation's standalone predictive performance on the training set. This works under the assumption that representations with stronger predictive performance contribute more valuable features. It is worth noting that this is not always the case, as high-quality, large representations may rely on a small subset of valuable features.

1. Train a RF on each base representation independently using the full training data.
2. Compute baseline R^2 for each representation via 5-fold cross-validation.
3. Allocate features proportionally:

$$n_r = B \times \frac{R_r^2}{\sum_{r'} R_{r'}^2}$$

4. For each representation, rank features by RF importance and select the top n_r.
5. Concatenate selected features to form the hybrid representation.

This method is a lot more computationally efficient than the greedy approach, however it cannot adapt allocation based on feature complementarity in the same way, and it relies on the assumption that many features in a well-performing representation are valuable.

5.3.3 Search configuration and internal evaluation

Features that improve predictions under a random forest may not help a kNN model, and vice versa. This is because variable importance depends fundamentally on the model, as Dong and Rudin showed [**?**]. To address this, the internal evaluator scores each candidate configuration with a model whose inductive bias matches the downstream model. We use five evaluators: a matched evaluator that uses the same model class as the downstream model, and four fixed evaluators with different

inductive biases (random forest, SVM, LightGBM, MLP). We fit each on the training fold and score it on the held-out fold. The matched evaluator ensures that features are selected jointly with the model that will use them.

The full search tests all combinations of budgets, allocations, importance methods, downstream models, and internal evaluators under 5-fold cross-validation: about 15,000 configurations per dataset when using 7 downstream models. This combinatorial design is deliberate: it allows us to measure which factors contribute most to performance variance (model choice, budget, allocation, or their interactions) rather than assuming any single axis dominates. We cap models sensitive to high dimensionality at 2,000 or 3,000 features depending on model type. The complete search result is cached to disk. The search runs once; everything after that is a lookup. Each search yields a Blueprint recording all parameters (source names, allocation strategy, importance method, budget, evaluator, metric) required to reproduce the hybrid on new data without re-running the search.

Figure 5.1: the search pipeline schematic. Does not exist yet.

5.4 Blueprints and explainability

Write Sec 5.4 from the outline bullets. No source prose exists.

5.5 Default settings

Adaptive pruning: allocation strategies are evaluated on fold 0 and pruned if they appear in the top-3 in fewer than a threshold fraction of (importance, evaluator, model, budget) groups. The primary hybrid searches (no augmentation and uncapped augmentation across general and top-tier representation sets) use a 15% threshold. Capped augmentation searches and standalone experiment-specific searches use a more aggressive 25% threshold to reduce computational cost; this higher threshold prunes more aggressively from the candidate allocation strategies without affecting the quality of the surviving strategies.

5.6 Evaluation metrics, models, and splitting

Different biological domains have established different conventions for what constitutes a good prediction, and applying a single metric everywhere would misrepresent performance in at least some of them. To this end, we assign each task a primary metric matched to its domain: RMSE for molecular property prediction, Spearman rank correlation for antibody regression and developability, AUROC for binary classification (including antibody classification and tumor subtype), weighted F1 for multiclass classification, concordance index for drug-target affinity, and R^2 for reaction yield.

For a fair comparison, the downstream model pool must cover a range of inductive biases, from methods that degrade in high dimensions (kNN, SVM, GP) to tree ensembles that tolerate irrelevant features. Thirteen models are available: tree ensembles (random forest, XGBoost, LightGBM, CatBoost, NGBoost), kernel methods (SVM, Gaussian process), nearest-neighbor (kNN), linear models (Ridge regression for regression, logistic regression for classification, both with cross-validated regularization), and neural networks (single-layer MLP, three-layer DNN, MC-Dropout BNN for uncertainty quantification, and a multi-scale 1D CNN). Standard experiments use seven of these: random forest, XGBoost, LightGBM, kNN, SVM, Gaussian process, and MLP. We use the full set for model-matching and noise experiments, and skip models with cubic or higher scaling for large datasets: Gaussian processes above 6,000 samples, SVMs above 10,000. No per-dataset tuning is performed: all models use fixed hyperparameters throughout, so performance differences reflect the input, not the model configuration.

No random splits are used. Random splitting inflates performance estimates for molecular data by placing structurally similar compounds on both sides of the train/test boundary [**??**], and position-level leakage in deep mutational scanning data allows models to memorise site-specific effects rather than learning generalisable sequence-fitness relationships [**??**]. Splits therefore match domain conventions and use 5-fold GroupKFold throughout.

The hybrid construction step uses a nested internal train/validation split within each training fold to score representations for feature allocation. This internal split is group-aware, using GroupShuffleSplit with the same grouping variable as the outer cross-validation, to prevent the allocation evaluator from seeing related samples on both sides of its split.

Comparing models tested under k-fold cross-validation requires care, because the training sets overlap between folds and a standard paired t-test overstates significance [?]. Within-dataset comparisons therefore use the Nadeau-Bengio corrected paired t-test, which adjusts for this dependence. Cross-dataset comparisons use the Wilcoxon signed-rank test on paired per-dataset scores, since performance differences need not be normally distributed over heterogeneous tasks. When ranking more than two methods, we apply the Friedman test with Nemenyi post-hoc correction [?] to control the family-wise error rate. We report effect sizes as rank-biserial correlation for Wilcoxon tests and Kendall's W for Friedman tests.

5.7 Implementation

RESULTS CAVEAT: state the re-run explicitly, once, here, and cross-reference it from ch.6 and ch.7.

5.8 Conclusions

Chapter 6

Why Hybrid Representations Help: Complexity, Explainability, and Extension

6.1 Introduction

A comparison of 62,820 models in molecular property prediction benchmarks showed that classical RDKit2D descriptors matched or outperformed learned features from graph neural networks and pretrained language models on the majority of datasets [Deng et al., 2023]. Descriptor-based models occupied 73% of top-three positions on 11 benchmarks [**?**], and in 1,310 ChEMBL assays, feedforward networks on ECFP fingerprints outperformed graph convolutions and SMILES-based recurrent models [**?**]. The input often matters more than the model. Yet we still lack a systematic way to exploit this. No single representation wins. The same pattern holds outside small-molecule chemistry: no single protein language model consistently achieves the highest performance on all tasks [**??**]; for antibodies, general-purpose models sometimes outperform those built for the domain [**?**]. In computational pathology, no individual foundation model dominates on clinical endpoints [**?**], and in cancer genomics, integrating additional omics layers can negatively impact performance when the data types are not properly combined [**?**].

For small molecules alone, the landscape now includes extended-connectivity fingerprints, physicochemical descriptor sets exceeding 1,600 dimensions, graph neural network embeddings [**?**Fang and Ye, 2022], language models trained on billions of SMILES strings [Ross et al., 2022], image-based encodings (molecules rendered as

2D images) [?], and stereoelectronics-infused molecular graphs [?]. The same is true for proteins (ESM-2 [?], ProtTrans [?]), antibodies (AntiBERTy [?], AbLang [?]), histopathology [?], and multi-omics cancer profiles [?]. A recent review counted over 40 molecular foundation models published between 2020 and 2024 [?]. Increasingly, new models appear faster than we can compare them [?]. The choice is not getting simpler. Different sources presumably encode distinct aspects of the same biology. Noisy or incomplete on their own, single data modalities somehow give more sensitive and robust predictions when combined with independent signals from other modalities [?]. An ensemble of frozen embeddings from the top four pathology foundation models, El Nahhas et al. found, achieved a mean AUROC 1.2% higher than the best individual model on 31 clinical tasks [?].

6.2 When it helps

6.2: Figures 6.1, 6.2; Tables 6.1, 6.2. Results pending re-run.

6.3 Why it helps

6.3: Figures 6.3, 6.4. Results pending re-run.

6.4 Explainability and stability

6.4: Figure 6.5; Table 6.3. Results pending re-run.

6.5 Breadth across domains

6.5: Figure 6.6. Results pending re-run.

6.6 Forward compatibility: absorbing new embeddings

6.6: Figure 6.7; Table 6.4. Results pending re-run.

6.7 Federated data (with Lhasa)

6.7: Figure 6.8; Tables 6.5, 6.6. Results pending re-run.

Concatenation, the most common method for combining feature sets, gives the full pool that KIRBy starts from, but on biological sample sizes it can degrade rather than help [??]. What late fusion gains by training separate models per source [?] it loses in cross-source feature interactions. Those interactions can be recovered through intermediate fusion (MOGONET [?], cross-attention [?]), but only at the cost of retraining whenever a new source appears. At the kernel level, multiple kernel learning [?] (e.g., pairwiseMKL for drug bioactivity [?]) can learn source weights, but no interpretable feature-level selections result. Consensus methods [?] combine predictions, not features, and so cannot exploit cases where one source's noise is another's signal. Instead, KIRBy works at the feature level: it selects features that span source boundaries, and because any new encoder's output is just another matrix in the pool, the approach is modular by construction. Perhaps the closest analogy in the optimization literature is the CASH problem [?] (the joint selection of algorithm and hyperparameters), extended here to include the input encoding as an additional search dimension. Tishby and Zaslavsky [?] showed why compression helps: the information bottleneck principle states that optimal representations maximize mutual information with the target while minimizing mutual information with the input. In effect, the budget forces the model to keep only what matters. The feature count controls the trade-off between informativeness and dimensionality.

Ambroise and McLachlan [?] showed the extreme case of feature selection overfitting: an SVM achieved near-zero cross-validated error on random labels when gene selection was performed on the full dataset before cross-validation, but when selection was nested inside each fold the median error was 11 out of 40, close to the 50% expected by chance. Vabalas et al. [?] showed that the bias introduced by preselection can exceed the bias from hyperparameter tuning, and Cawley and Talbot [?] argued that feature selection is itself a form of model selection whose complexity cost must be accounted for. KIRBy follows the standard remedy: we compute all importance scores within each cross-validation fold, on training data only. But the problem of "search degrees of freedom" [?] remains. Searching over many possible feature subsets adds complexity even when each individual selection is unbiased. The budget acts as a hard capacity constraint: selecting 100 features from a pool of 5,000 limits the space of possible selections far more tightly than L_1 regularization

over the same pool, because the feasible set grows polynomially in the budget while the continuous parameter space of a regularized model is unbounded [?]. Important here is a distinction: the features being selected are frozen embeddings, not parameters being fitted. Choosing which pre-existing knowledge to trust is, we think, a less expressive operation than learning new features, and Kumar et al. [?] showed that frozen embeddings fed through a linear probe can outperform full fine-tuning by up to 10% in out-of-distribution accuracy precisely because fine-tuning distorts pretrained structure. And stability can be measured directly: the Nogueira index [?] quantifies the consistency of selected feature sets between folds. If selection were overfit, feature sets would differ wildly from fold to fold.

6.8 Conclusions

Chapter 7

Molecular Similarity and Label Noise

7.1 Introduction

7.2 Similarity and the representation

7.2.1 Distance metrics for continuous molecular descriptors

The choice of distance, or similarity metric has a strong impact on performance of distance-based noise estimation methods. We evaluate eight distance metrics appropriate for continuous molecular descriptors:

- Euclidean (L_2): Standard geometric distance, $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$
- Manhattan (L_1): Sum of absolute differences, $d(x, y) = \sum_i |x_i - y_i|$, more robust to outlier features
- Cosine: Angle-based similarity, $d(x, y) = 1 - \frac{x \cdot y}{\|x\|\|y\|}$, invariant to magnitude
- Chebyshev (L_∞): Maximum absolute difference, $d(x, y) = \max_i |x_i - y_i|$
- Canberra: Weighted L_1 normalized by magnitude, $d(x, y) = \sum_i \frac{|x_i - y_i|}{|x_i| + |y_i|}$
- Correlation: $d(x, y) = 1 - \rho(x, y)$, where ρ is the Pearson correlation
- Bray-Curtis: Ecological dissimilarity measure, $d(x, y) = \frac{\sum_i |x_i - y_i|}{\sum_i |x_i + y_i|}$
- Mahalanobis: Covariance-adjusted distance accounting for feature correlations

All features are standardised (zero mean, unit variance) before distance computation to ensure equal contribution across features with different relative scales.

7.2.2 Feature importance weighting

Beyond simply using the feature importance scores for allocation, we investigate whether feature weighting by importance during distance computation improves noise detection. For a feature importance vector w, weighted distances are computed by scaling features: $x' = x \odot \sqrt{w}$.

7.3 Noise estimation and mitigation inside the method

7.3.1 Sample removal methods

Initially, I attempted to adapt noise mitigation techniques that focused on removing noisy samples. However given that in these experiments, I'm adding noise to all of the data, this task because not just difficult but impractical as all samples are noisy, it's more about understanding the magnitude of noise. Therefore, I shifted my to sample weighting and label smoothing which I believed would be better suited for handling Gaussian noise affecting all samples.

Distance-based approaches rely on the assumption that similar molecules have similar properties [Cherkasov et al., 2014]. Presumably, noisy data would break this pattern, making it a method for detecting anomalous data. k-Nearest neighbor filtering examines the variance of a sample's neighborhood. For sample x_i with k nearest neighbors $\mathcal{N}_k(x_i)$, we compute the standard deviation of neighbor labels:

$$\sigma_i = \text{std}(\{y_j : x_j \in \mathcal{N}_k(x_i)\})$$

Samples with σ_i exceeding the 90th percentile are marked as potentially noisy. We test $k \in \{5, 10, 20\}$.

Local Outlier Factor (LOF) identifies anomalous samples by detecting local density estimates between a sample and its neighbors [Breunig et al., 2000]. These density estimates are based on the distance to its k nearest neighbors. If a point's density differs from that of its neighbors such that the LOF is substantially greater than 2, it is flagged as an outlier.

We also tested the following noise removal methods: distance-weighted prediction filtering, neighborhood consensus filtering, AC detection, Mahalanobis outlier detection, cross-validation disagreement, bootstrap ensemble filtering, co-teaching [Han et al., 2018], and DivideMix [Li et al., 2020].

7.3.2 Noise magnitude estimation

We now examine the core of the noise mitigation research: estimating the magnitude of noise. This enables proportional handling rather than complete removal. We used the following neighbor-based estimators:

- Neighbor residual: Estimates noise as deviation from k-NN mean, normalized by global label standard deviation:

$$\text{score}_i = \frac{|y_i - \bar{y}_{\mathcal{N}_k(x_i)}|}{\sigma_y}$$

- Local variance ratio: Measures deviation relative to local label variance, identifying samples that deviate from neighbors more than neighbors deviate from each other:

$$\text{score}_i = \frac{|y_i - \bar{y}_{\mathcal{N}_k(x_i)}|}{\text{std}(y_{\mathcal{N}_k(x_i)}) + \epsilon}$$

- AC-aware: Distinguishes noise from activity cliffs by weighting deviation by neighborhood consistency. High deviation with high neighbor agreement suggests noise while high deviation with neighbor disagreement suggests an AC [Dablander et al., 2023]:

$$\text{score}_i = \frac{|y_i - \bar{y}_{\mathcal{N}_k(x_i)}|}{\sigma_y} \cdot \frac{1}{1 + \text{std}(y_{\mathcal{N}_k(x_i)})/\sigma_y}$$

And we used the following model-based estimators using prediction disagreement:

- Cross-validation residual: Uses out-of-fold predictions; high residuals indicate labels inconsistent with the model learned from other samples.

- Ensemble disagreement: Trains multiple models and combines prediction variance with mean residual.

We used probabilistic methods that directly predict noise magnitude using uncertainty estimates:

- Gaussian Process: Provides natural uncertainty estimates via posterior variance, with learned noise level σ^2 from the kernel [Griffiths and et al., 2022].

- Heteroscedastic Neural Network: Predicts both $\mu(x)$ and $\sigma^2(x)$ using β-NLL loss [Seitzer et al., 2022]:

$$\mathcal{L} = \frac{1}{2}\left(\frac{(y-\mu)^2}{\sigma^2} + \log \sigma^2\right) \cdot \sigma^{2\beta}$$

where $\beta = 0.5$ prevents the variance from hiding the mean errors.

- Conformal prediction: Uses split conformal prediction to obtain prediction intervals. Wider intervals should indicate higher uncertainty [Shafer and Vovk, 2008].
- Deep ensemble: Trains M independent networks and uses the variance of means as epistemic uncertainty plus the mean of variances as aleatoric uncertainty [Lakshminarayanan et al., 2017].
- Bayesian Random Forest: Uses tree prediction variance and out-of-bag residuals as uncertainty estimates [Taddy et al., 2015].
- Quantile regression: Fits models for multiple quantiles and uses interval width as noise estimate.
- NGBoost: Natural gradient boosting that predicts distributional parameters (μ, σ) directly [Duan et al., 2020].

NGBoost in particular was selected due to its promising uncertainty values with respect to the correlation between noise and prediction error, as discussed in Chapter 3.

7.3.3 Label smoothing and sample weighting

Given the fact that all labels contain some magnitude of noise, we focused strongly on label smoothing over removal using the following methods:

- Proportional: $y_{\text{smooth}} = y + \alpha \cdot \text{score}_{\text{norm}} \cdot (\text{target} - y)$
- Threshold: Only correct samples above the $(1 - \alpha)$ percentile of scores.
- Soft threshold: Applies sigmoid-like correction strength:

$$\text{strength} = \alpha \cdot \sigma\left(5 \cdot (\text{score}_{\text{norm}} - 0.5)\right)$$

- Confidence: For methods that output raw uncertainty σ (e.g., NGBoost):

$$\text{correction} = \frac{\alpha \cdot \sigma}{1 + \alpha \cdot \sigma}, \quad y_{\text{smooth}} = (1 - \text{correction}) \cdot y + \text{correction} \cdot \mu$$

Rather than directly modifying labels, we weighted more trustworthy samples during training:

- Inverse: $w = \frac{1}{1+\alpha \cdot \text{score}_{\text{norm}}}$
- Exponential: $w = \exp(-\alpha \cdot \text{score}_{\text{norm}})$
- Soft: $w = 1 - \alpha \cdot \text{score}^2_{\text{norm}}$, clipped to $[0.1, 1.0]$

All weights are normalized to mean 1 to preserve effective sample size.

7.3.4 Recovery rate

For noise detection accuracy, we compute precision, recall, and F1 score, treating noise injection as ground truth.

We have a standard RF on ECFP4 train and test on the clean dataset, producing R^2_{clean}. Noise is then added to the clean data, RF on ECFP4 is then freshly trained and tested on this noisy data, producing R^2_{noisy}. Using the given estimation method and representation to test, noise estimation and label smoothing is applied to the noisy data, resulting in the denoised set. RF on ECFP4 is yet again run, this time on the denoised data, to produce R^2_{denoised}. From these performance values, we derive recovery rate, or the fraction of performance loss recovered after denoising, as follows:

$$\text{Recovery} = \frac{R^2_{\text{denoised}} - R^2_{\text{noisy}}}{R^2_{\text{clean}} - R^2_{\text{noisy}}}$$

A recovery rate of 1.0 indicates complete restoration to clean performance; negative values indicate the denoising procedure made performance worse than the noisy baseline.

7.4 Conclusions

Overall, we see in agreement with conclusions of Chapter 4 on the use of distance metrics in LNL methods, the concept of using a sample's neighbors to determine it's ground truth is flawed. It is likely all labels are too uniformly noisy for a methodology like this. Uncertainty-based methods show potential at lower noise, but their estimation performance degrades with higher noise. Sampling weighting methods generally proved effective, and one estimator using exponential sample weighting produced very strong recovery across both noise levels.

Hybrid representations had mixed results. For sample removal, hybrids showed advantages at high noise, but struggled at low noise. They showed consistently

positive recovery rates for neighbor-based estimation, which roughly falls in line with the idea that a low-dimension, high-quality information representation benefits neighbor-based reasoning. The feature budget in this experiment was restricted to 50-100 features, which may limit its advantages. This range was successful at standard molecular property prediction, but potentially not for certain noise mitigation or estimation tasks. Distance-based methods remain challenging under Gaussian noise regardless of representation, suggesting the core issue is the noise distribution rather than representation quality alone. This merits an investigation into mitigation of noise from other distributions or noise injection strategies.

Chapter 8

Discussion and Future Directions

8.1 Conclusions

Write Sec 8.1. Draw on [NI 586], already used to close Sec 4.6 – do not simply repeat it.

8.2 Limitations

We want to be clear about KIRBy's limitations. For small datasets (below about 200 samples), the hybrid search cannot reliably outperform the best single source. We suspect this is because the internal validation set within each fold shrinks to roughly 20 samples, and at that size the evaluator cannot distinguish genuine improvements from noise. Tuning cannot fix this. And at the other extreme, frozen embeddings have a ceiling. No signal can be added beyond what the inputs already contain, and on tasks where the true signal requires end-to-end learning from raw inputs (certain 3D quantum-mechanical properties, for example, where SchNet outperforms any frozen embedding by a wide margin), this ceiling binds. Although the methodology naturally accommodates embeddings from sequence-trained models (ESM-2, ProtBERT, etc.), encoding raw sequences directly as one-hot or k-mer features disperses the positional and compositional information that makes sequences useful in the first place. Integrating 20 to 50 foundation models into a single pipeline, each with its own dependencies, GPU requirements, and version constraints, is genuinely difficult engineering. Embeddings need only be computed once per dataset and everything is cached. Yet the initial integration burden is real and scales with the number of models in the pool. Finally, on endpoints with high experimental noise,

even a well-constructed hybrid may not improve on simpler baselines. Kramer et al. [Kramer et al., 2012] documented that experimental uncertainty in heterogeneous public K_i data is approximately 0.54 log units. This sets a natural accuracy ceiling of $R^2 \approx 0.81$ regardless of model sophistication; the budget constraint plausibly guards against overshoot, but it cannot push past the noise floor.

8.3 Future directions

I also plan to extend the hybrid research to additional base representations and datasets. Some alternative representations include additional GNN embeddings, sequential encoders, augmented features such as embedded distances from molecular graphs, and transformed representations (i.e., normalization, binarization of continuous features). Additionally, I want to explore alternative feature importance scoring methods, especially ones that involve different families of models. Specifically, I plan to look at TreeSHAP, KernelSHAP, drop-column importance for noise robustness, gradient-based attribution for NN embeddings, and small-loss selection, as discussed in Chapter 4.

As mentioned earlier, although I do not currently have a full set of results on hybrid, they have been showing promise throughout a rank of QSAR and noise mitigation tasks. I would like to explore the complexity tradeoff between model and representation complexity, as I've seen a trend of complicated models like NNs having stronger predictive performance with smaller hybrids and more simplistic tree-based methods benefiting from larger hybrids. This noise mitigation experiment relied on a very small default hybrid and used RF-based feature importance. I would like to leverage my allocation strategies to adapt hybrid models to datasets *and* models. These strategies can also be modified to optimize for noise robustness in addition to predictive performance.

Various estimation methods showed promise, meriting further investigation. Additionally, neighbor-based noise mitigation strategies which are not fit for Gaussian noise may perform differently on outlier-based strategies. The dominance of estimators with label smoothing and sample weighting may be challenged for different types of noise.

What does this mean in practice? A new foundation model appears every few months in molecular, protein, or antibody modeling. A researcher using KIRBy does not need to decide whether to replace their current encoding or retrain their

pipeline; they compute the new model's embeddings, add the matrix to the pool, and re-run the search. The search determines whether the new embedding contributes useful features and, if so, how many and which ones. In a field where encoding choice is often determined by familiarity or lab convention rather than by systematic comparison, this is a substantive change. It replaces a single bet with a principled combination, tailored to each task, that updates as the pool grows.

Summary of Research Contributions

8.4 Research Papers

In prepration: Punt, A., Morris, G., Werner, S., & Hanser, T. (2026). Noise-Robust QSAR: evaluating probabilistic methods across molecular representations
My supervisors and I are currently iterating on an advanced draft for this research paper based on the material in Chapter 3, which we intend to submit to the *Journal of Cheminformatics* within the next few months.

In prepration: Punt, A., Morris, G., Werner, S., & Hanser, T. (2026). Improving noise prediction robustness and mitigation with feature-importance hybrid representations in multi-modal contexts
I am in the early stages of writing a draft and extending the experiments in Chapter 5 within the next few months.

8.5 Conference Presentations

Punt, A., Morris, G., Werner, S., & Hanser, T. (2024). Investigating the impact and mitigation of noise in QSAR models. Posted presented at the Autumn UK-QSAR 2024 Meeting.

8.6 Blogposts

I have contributed to the *Blopig* blog of the Oxford Protein Informatics Group (OPIG):

- Are you addicted to dopamine?
- Demystifying Git and merge conflicts
- AI in Academic Writing: Ethical Considerations and Best Practices

- Memory-mapped files for efficient data processing
- Exploring multilingual programming

Appendix A

Datasets and curation

The datasets used in this study are publicly available. The QM9 dataset is available on Figshare [Ramakrishnan et al., 2014a] and is also redistributed through the OpenADMET benchmark suite [Ramakrishnan et al., 2014b, Open ADMET Consortium, 2026]. The LogD and Caco-2 datasets are available in the OpenADMET repository [Open ADMET Consortium, 2026]. The hERG-Ki dataset was derived from ChEMBL [Zdrazil et al., 2023] following the curation protocol described in Section 2, and can be regenerated from ChEMBL using that protocol.

Appendix A: add per-dataset provenance, accession numbers, DOIs.

Appendix B

NoiseInject supplementary material

B.1 Default hyperparameters for all models evaluated in this study

Appendix B.1: was Additional file 1 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:668.

B.2 Pairwise Spearman rank correlations between model NDS profiles

Appendix B.2: was Additional file 2 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:669.

B.3 Pairwise Spearman rank correlations between representation NDS profiles

Appendix B.3: was Additional file 3 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:670.

B.4 Intraclass correlation coefficients (ICC(1,1)) for model pairs

Appendix B.4: was Additional file 4 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:671.

B.5 Configurations excluded from robustness analysis

Appendix B.5: was Additional file 5 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:672.

B.6 Global overview of noise robustness on the ECFP4 representation

Appendix B.6: was Additional file 6 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:673.

B.7 Baseline R^2 versus NDS scatter (PDV, Gaussian)

Appendix B.7: was Additional file 7 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:674.

B.8 Neural-network Bayesian transformation effects by representation

Appendix B.8: was Additional file 8 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:675.

B.9 Uncertainty quantification metrics for probabilistic models on the ECFP4 representation

Appendix B.9: was Additional file 9 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:676.

B.10 ANOVA variance decomposition on external validation datasets

Appendix B.10: was Additional file 10 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:677.

B.11 RF vs QRF on validation datasets

Appendix B.11: was Additional file 11 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:678.

B.12 Default hyperparameters used for external validation experiments

Appendix B.12: was Additional file 12 of the NoiseInject manuscript. Titles carried verbatim from noise_inject.tex:679.

Appendix C

The search space and its defaults

Appendix C is unwritten.

Appendix D

Extended result tables

Appendix D is unwritten.

Appendix E

Data and code availability

The NoiseInject benchmarking framework, implementing all six regression and six classification noise injection strategies together with the robustness metrics (noise degradation slope and retention) and uncertainty-calibration metrics (ECE, coverage at $1\sigma/2\sigma$, mean interval width, and uncertainty–error / uncertainty–noise correlation) described in this work, is available as an open-source Python package under an MIT license [Punt, 2026]. It also contains optional reference wrappers (a Gaussian process, split-conformal prediction, and Monte Carlo dropout) that produce per-sample uncertainty for use with these metrics. NoiseInject is model-agnostic, operates on label arrays (NumPy arrays or pandas Series), and is designed to integrate into existing cheminformatics or other data pipelines. Worked examples covering molecular-descriptor, fingerprint, and GNN-embedding workflows are provided in the repository.

- Project name: NoiseInject
- Project home page: `https://github.com/adpunt/noise_inject`
- Archived version: `https://doi.org/10.5281/zenodo.20532710`
- Operating system(s): Platform independent
- Programming language: Python
- Other requirements: See repository `requirements.txt` for the full dependency list
- License: MIT

- Any restrictions to use by non-academics: None

A.P. implemented the NoiseInject framework, conducted all experiments, and drafted the manuscript. T.H., S.W., and G.M. supervised the project, contributed to the experimental design, and revised the manuscript. All authors read and approved the final manuscript.

Appendix E: add itemised blocks for the Rust/Python precompute pipeline and for KIRBy, plus the Lhasa/Effiris redistribution caveat.

Bibliography

Walid Ahmad, Elana Simon, Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta-2: Towards chemical foundation models. *arXiv preprint arXiv:2209.01712*, 2022.

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 2623–2631, 2019.

Ismail Babajide Mustapha and Faisal Saeed. Bioactive molecule prediction using extreme gradient boosting. *Molecules*, 21(8), 2016. ISSN 1420-3049. doi: 10.3390/molecules21080983.

Michael Backenköhler, Paula Linh Kramer, and Joschka et al. Groß. TeachOpenCADD goes Deep Learning: Open-source Teaching Platform Exploring Molecular DL Applications. *ChemRxiv preprint*, 2023. doi: 10.26434/chemrxiv-2023-kz1pb.

Dávid Bajusz, Anita Rácz, and Károly Héberger. Why is tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7(1), May 2015. ISSN 1758-2946. doi: 10.1186/s13321-015-0069-3.

Igor I. Baskin, Vladimir A. Palyulin, and Nikolai S. Zefirov. Neural networks in building QSAR models. In *Methods in Molecular Biology*, volume 458, pages 137–158. Humana Press, 2008. doi: 10.1007/978-1-60327-101-1_8.

Davide Bassani, Alessandro Brigo, and Andrea Andrews-Morger. Federated learning in computational toxicology: An industrial perspective on the effiris hackathon. *Chemical Research in Toxicology*, 36(9):1503–1517, 2023.

A Patrícia Bento, Anne Hersey, Eloy Félix, and Greg et al. Landrum. An open source

chemical structure curation pipeline using RDKit. *Journal of Cheminformatics*, 12(1):51, 2020. doi: 10.1186/s13321-020-00456-1.

Martino Bertoni, Miquel Duran-Frigola, Pau Badia-i Mompel, Eduardo Pauls, Modesto Orozco-Ruiz, Oriol Guitart-Pla, Víctor Alcalde, Víctor M Diaz, Antoni Berenguer-Llergo, Antonio García de Herreros, and Patrick Aloy. Bioactivity descriptors for uncharacterized compounds. *bioRxiv*, 2020. doi: 10.1101/2020.07.21.214197.

Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1):657–668, 2023.

Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Monterey, CA, 1984. ISBN 978-0-412-04841-8.

Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, page 93–104, New York, NY, USA, 2000. Association for Computing Machinery. ISBN 1581132174. doi: 10.1145/342009.335388.

Christine Brideau, Bert Gunter, Bill Pikounis, and Andy Liaw. Improved statistical methods for hit selection in high-throughput screening. *Journal of Biomolecular Screening*, 8(6):634–647, 2003. doi: 10.1177/1087057103258285.

Frank Burden and Dave Winkler. *Bayesian Regularization of Neural Networks*, pages 23–42. Humana Press, Totowa, NJ, 2009. ISBN 978-1-60327-101-1. doi: 10.1007/978-1-60327-101-1_3.

Alexander Camuto, Matthew Willetts, Umut Şimşekli, Stephen Roberts, and Chris Holmes. Explicit regularisation in gaussian noise injections, 2021.

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

Artem Cherkasov, Eugene N. Muratov, and Denis et al. Fourches. QSAR modeling: Where have you been? where are you going to? *Journal of Medicinal Chemistry*, 57(12):4977–5010, 2014. doi: 10.1021/jm4004285.

Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta: Large-scale self-supervised pretraining for molecular property prediction. *arXiv preprint arXiv:2010.09885*, 2020.

Isidro Cortes-Ciriano, Andreas Bender, and Thérèse E. Malliavin. Comparing the influence of simulated experimental errors on 12 machine learning algorithms in bioactivity modeling using 12 diverse data sets. *Journal of Chemical Information and Modeling*, 55(7):1413–1425, 2015. doi: 10.1021/acs.jcim.5b00101. PMID: 26038978.

Isidro Cortés-Ciriano, Ctibor Škuta, Andreas Bender, and Daniel Svozil. QSAR-derived affinity fingerprints (part 2): Modeling performance for potency prediction. *Journal of Cheminformatics*, 12(1):41, 2020.

Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Sort & slice: a simple and superior alternative to hash-based folding for extended-connectivity fingerprints. *Journal of Cheminformatics*, 16(1), December 2024. ISSN 1758-2946. doi: 10.1186/s13321-024-00932-y.

Michael Dablander, Thierry Hanser, Renaud Lambiotte, et al. Exploring QSAR models for activity-cliff prediction. *Journal of Cheminformatics*, 15(1):47, 2023.

Luís David, Abhijit Thakkar, Raul Mercado, et al. Molecular representations in AI-driven drug discovery: a review and practical guide. *Journal of Cheminformatics*, 12(1):56, 2020.

Jianyuan Deng, Zhibo Yang, and Hehe Wang et al. Unraveling key elements underlying molecular property prediction: A systematic study, 2023.

Wenjie Du, Xiaoting Yang, and Di et al. Wu. Fusing 2D and 3D molecular graphs as unambiguous molecular descriptors for conformational and chiral stereoisomers. *Briefings in Bioinformatics*, 24(1):bbac560, 2022. ISSN 1477-4054.

Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. Ngboost: Natural gradient boosting for probabilistic prediction, 2020.

Benedek Fabian, Thomas Edlich, and Héléna et al. Gaspar. Molecular representation learning with language models and domain-relevant auxiliary tasks. *arXiv preprint arXiv:2011.13230*, 2020.

Xiuwen Fang and Mang Ye. Robust federated learning with noisy and heterogeneous clients. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10062–10071, 2022.

Alexandre V. Fassio, Laura Shub, and Luca Ponzoni et al. Prioritizing virtual screening with interpretable interaction fingerprints. *Journal of Chemical Information and Modeling*, 62(18):4300–4318, 2022.

Artem Fediai et al. Accurate GW frontier orbital energies of 134 kilo molecules. *Scientific Data*, 10:581, 2023. doi: 10.1038/s41597-023-02486-4.

Matthias Fey and Jan E. Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and qsar modeling research. *Journal of Chemical Information and Modeling*, 50(7):1189–1204, 2010. doi: 10.1021/ci100176x. PMID: 20572635.

Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify II: A practical guide to chemogenomics data curation. *Journal of Chemical Information and Modeling*, 56(7):1243–1252, 2016. doi: 10.1021/acs.jcim.6b00129.

Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning, 2016.

Anna Gaulton, Anne Hersey, and Michał et al. Nowotka. The ChEMBL database in 2017. *Nucleic Acids Research*, 45(D1):D945–D954, 2017. doi: 10.1093/nar/gkw1074.

Michael K Gilson, Tiqing Liu, Michael Baitaluk, and et al. BindingDB in 2015: A public database for medicinal chemistry, computational chemistry and systems pharmacology. *Nucleic Acids Research*, 44(D1):D1045–D1053, 2016. doi: 10.1093/nar/gkv1072.

Ryan-Rhys Griffiths and Leo Klarner et al. Gauche: A library for gaussian processes in chemistry, 2022. Accessed: 2024-08-21.

Zhichun Guo, Kehan Guo, and Bozhao Nan et al. Graph-based molecular representation learning, 2023. Accessed: 2024-11-30.

Bo Han, Quanming Yao, and Xingrui Yu et al. Co-teaching: Robust training of deep neural networks with extremely noisy labels. 2018. Accessed: 2024-09-01.

Thierry Hanser. Federated learning for molecular discovery. *Current Opinion in Structural Biology*, 79:102545, 2023. ISSN 0959-440X.

James Harrison, John Willes, and Jasper Snoek. Variational bayesian last layers, 2024.

Christos Hatzis, Philippe L. Bedard, Nicolai J. Birkbak, Andrew H. Beck, Hugo J.W.L. Aerts, David F. Stern, Leming Shi, Robert Clarke, John Quackenbush, and Benjamin Haibe-Kains. Enhancing reproducibility in cancer drug screening: How do we move forward? *Cancer Research*, 74(15):4016–4023, July 2014. ISSN 1538-7445. doi: 10.1158/0008-5472.can-14-0725.

Esther Heid, Charles J. McGill, Florence H. Vermeire, and William H. Green. Characterizing uncertainty in machine learning for chemistry. *Journal of Chemical Information and Modeling*, 63(13):4012–4029, 2023. doi: 10.1021/acs.jcim.3c00373. PMID: 37338239.

Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

Eyke Hüllermeier and Willem Waegeman. Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods. *Machine Learning*, 110(3):457–506, 2021. ISSN 1573-0565.

Stefan M.N. Hönig, Carsten Lemmen, and Matthias Rarey. Small molecule superposition: A comprehensive overview on pose scoring of the latest methods. *WIREs Computational Molecular Science*, 13(2):e1640, 2023.

James Inglese, Douglas S. Auld, and Ajit et a.. Jadhav. Quantitative high-throughput screening: A titration-based approach that efficiently identifies biological activities in large chemical libraries. *Proceedings of the National Academy of Sciences*, 103(31):11473–11478, 2006. doi: 10.1073/pnas.0604348103.

Mohammad Jahidul Islam, Ajoy Kumer, Nuruzzaman Sarker, Sunanda Paul, and Afroza Zannat. The prediction and theoretical study for chemical reactivity, thermophysical and biological activity of morpholinium nitrate and nitrite ionic liquid crystals: A dft study. *Advanced Journal of Chemistry, Section A*, 2(4):316–326, 2019. ISSN 2645-7768. doi: 10.33945/SAMI/AJCA.2019.4.5.

Sabrina Jaeger, Simone Fulle, and Samo Turk. Mol2vec: Unsupervised machine learning approach with chemical intuition. *Journal of Chemical Information and Modeling*, 58(1):27–35, 2018a. doi: 10.1021/acs.jcim.7b00616.

Sabrina Jaeger, Simone Fulle, and Samo Turk. Mol2vec: Unsupervised machine learning approach with chemical intuition. *Journal of Chemical Information and Modeling*, 58(1):27–35, 2018b. doi: 10.1021/acs.jcim.7b00616.

Lu Jiang, Zhengyuan Zhou, Thomas Leung, Li-Jia Li, and Li Fei-Fei. Mentornet: Learning data-driven curriculum for very deep neural networks on corrupted labels, 2018.

Kjell Jorner, Tore Brinck, Per-Ola Norrby, and David Buttar. Machine learning meets mechanistic modelling for accurate prediction of experimental activation energies. *Chem. Sci.*, 12:1163–1175, 2021. doi: 10.1039/D0SC04896H.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154, 2017.

Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? *CoRR*, abs/1703.04977, 2017.

Kristian Kersting, Christian Plagemann, Patrick Pfaff, and Wolfram Burgard. Most likely heteroscedastic gaussian process regression. In *Proceedings of the 24th International Conference on Machine Learning*, ICML '07, page 393–400, New York, NY, USA, 2007. Association for Computing Machinery. ISBN 9781595937933. doi: 10.1145/1273496.1273546.

Akihiro Kishimoto, Hiroshi Kajino, Masataka Hirose, Junta Fuchiwaki, Indra Priyadarsini, Lisa Hamada, Hajime Shinohara, Daiju Nakano, and Seiji Takeda. Mhg-gnn: Combination of molecular hypergraph grammar with graph neural network, 2023.

Scott S. Kolmar and Christopher M. Grulke. The effect of noise on the predictive limit of QSAR models. *Journal of Cheminformatics*, 13(1):92, 2021.

Alexios Koutsoukas, Keith J. Monaghan, Xiaoli Li, and Jun Huan. Deep-learning: investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data. *Journal of Cheminformatics*, 9:42, 2017. doi: 10.1186/s13321-017-0226-y.

Christian Kramer, Tuomo Kalliokoski, Peter Gedeck, and Anna Vulpetti. The experimental uncertainty of heterogeneous public Ki data. *Journal of Medicinal Chemistry*, 55(11):5165–5173, 2012. doi: 10.1021/jm300131x.

Denis Kuzminykh, Daniil Polykovskiy, and Artur Kadurin et al. 3D molecular representations based on the wave transform for convolutional neural networks. *Molecular Pharmaceutics*, 15(10):4378–4385, 2018.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles, 2017.

Greg Landrum. Rdkit: Open-source cheminformatics. https://www.rdkit.org. Accessed: 2026-01-09.

Gregory A. Landrum and Sereina Riniker. Combining IC50 or Ki values from different sources is a source of significant noise. *Journal of Chemical Information and Modeling*, 64(5):1560–1567, 2024. doi: 10.1021/acs.jcim.4c00049. PMID: 38394344.

Oliver Laufkötter, Noé Sturm, Jürgen Bajorath, Hongming Chen, and Ola Engkvist. Combining structural and bioactivity-based fingerprints improves prediction performance and scaffold hopping capability. *Journal of Cheminformatics*, 11(1), August 2019. ISSN 1758-2946. doi: 10.1186/s13321-019-0376-1.

Junnan Li, Richard Socher, and Steven C. H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning, 2020.

Markus A. Lill. Multi-dimensional QSAR in drug discovery. *Drug Discovery Today*, 12(23):1013–1017, 2007. ISSN 1359-6446.

Haitao Liu, Yew-Soon Ong, and Jianfei Cai. Large-scale heteroscedastic regression via gaussian process, 2020.

Junshui Ma, Robert P Sheridan, Andy Liaw, George E Dahl, and Vladimir Svetnik. Deep neural nets as a method for quantitative structure-activity relationships. *Journal of Chemical Information and Modeling*, 55(2):263–274, 2015. doi: 10.1021/ci500747n.

Ricardo Macarron, Martyn N. Banks, and Dejan et al. Bojanic. Impact of high-throughput screening in biomedical research. *Nature Reviews Drug Discovery*, 10 (3):188–195, 2011. doi: 10.1038/nrd3368.

Andrea Mauri, Viviana Consonni, Manuela Pavan, and Roberto Todeschini. DRAGON software: An easy approach to molecular descriptor calculations. *MATCH Communications in Mathematical and in Computer Chemistry*, 56(2): 237–248, 2006.

Nicolai Meinshausen. Quantile regression forests. *Journal of Machine Learning Research*, 7:983–999, 06 2006.

Henry Moss and Ryan-Rhys Griffiths. Gaussian process molecule property prediction with flowmo, 10 2020.

Bruno J. Neves, Rodolpho C. Braga, and Cleber C. Melo-Filho et al. QSAR-based virtual screening: Advances and applications in drug discovery. *Frontiers in Pharmacology*, 9:1275, 2018. ISSN 1663-9812.

Curtis G Northcutt, Lu Jiang, and Isaac L Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021a. doi: 10.1613/jair.1.12125.

Curtis G Northcutt, Lu Jiang, and Isaac L Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021b.

Olga Obrezanova, Joelle M.R. Gola, and Matthew D. Segall. Gaussian processes: A method for automatic qsar and adme modelling. Presentation at ACS Boston, 22 August 2007, 2007.

Open ADMET Consortium. OpenADMET-ExpansionRx challenge dataset, 2026. URL https://huggingface.co/datasets/openadmet/openadmet-expansionrx-challenge-data. CC-BY-4.0. ADMET data from drug discovery campaigns by Expansion Therapeutics targeting RNA-mediated diseases. Challenge closed January 19, 2026.

Glenn Palmer, Siqi Du, Alexander Politowicz, Joshua Emory, Xiyu Yang, Anupraas Gautam, Grishma Gupta, Zhelong Li, Ryan Jacobs, and Dane Morgan. Calibration after bootstrap for accurate uncertainty quantification in regression models. *npj Computational Materials*, 8:115, 05 2022. doi: 10.1038/s41524-022-00794-8.

Elena Postnikova, Yu Cong, Lisa Evans DeWald, and Julie et al. Dyall. Testing therapeutics in cell-based assays: Factors that influence the apparent potency of drugs. *PLOS ONE*, 13(3):e0194880, March 2018. ISSN 1932-6203. doi: 10.1371/journal.pone.0194880.

Adelaide Punt. NoiseInject: A Python package for label-noise robustness benchmarking in QSAR models. https://github.com/adpunt/noise_inject, 2026. Accessed: 2026-06-01.

Joaquin Quiñonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate gaussian process regression. *Journal of Machine Learning Research*, 6(65):1939–1959, 2005.

Swamidass S. Joshua Ralaivola, L., Hiroto Saigo, and Pierre Baldi. Graph kernels for chemical informatics. *Neural networks : the official journal of the International Neural Network Society*, 18:1093–110, 11 2005. doi: 10.1016/j.neunet.2005.07.009.

Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules

(QM9 dataset). https://doi.org/10.6084/m9.figshare.978904, 2014a. Accessed: 2026-06-02.

Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, et al. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014b.

Bharath Ramsundar and Peter Eastman et al. Deep learning for the life sciences, 2019. Accessed: 2024-09-01.

Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 2005. ISBN 9780262256834.

David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t. PMID: 20426451.

Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties, 2022.

María Virginia Sabando, Ignacio Ponzoni, Evangelos E Milios, and Axel J Soto. Using molecular embeddings in QSAR modeling: Does it make a difference? *Briefings in Bioinformatics*, 23(1):bbab365, 2022. ISSN 1477-4054.

Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks, 2022.

Glenn Shafer and Vladimir Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9(12):371–421, 2008.

Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-weight-net: Learning an explicit mapping for sample weighting, 2019.

Anton Simeonov, Ajit Jadhav, and Craig J. et al. Thomas. Fluorescence spectroscopic profiling of compound libraries. *Journal of Medicinal Chemistry*, 51(8): 2363–2371, 2008. doi: 10.1021/jm701301m.

Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8135–8153, 2023. doi: 10.1109/TNNLS.2022.3152527.

Vladimir Svetnik, Andy Liaw, Christopher Tong, John Culberson, Robert Sheridan, and Bradley Feuston. Random forest: A classification and regression tool for compound classification and qsar modeling. *Journal of chemical information and computer sciences*, 43:1947–58, 11 2003. doi: 10.1021/ci034160g.

Matt Taddy, Chun-Sheng Chen, Jun Yu, and Mitch Wyle. Bayesian and empirical bayesian forests, 2015.

Hao Tian, Rajas Ketkar, and Peng Tao. Accurate admet prediction with xgboost, 2022.

Roberto Todeschini and Viviana Consonni. *Molecular Descriptors for Chemoinformatics*. Wiley-VCH, Weinheim, 2nd edition, 2009. doi: 10.1002/9783527628766.

Viktoriia Turkina, Melanie R.W. Messih, Etienne Kant, and Jelle T. et al. Gringhuis. Enhancing predictive modeling with molecular fingerprint fusion strategies. July 2025. doi: 10.26434/chemrxiv-2025-b3lfq.

Insha Ullah and A. H. Welsh. On the effect of noise on fitting linear regression models, 2024.

Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995. doi: 10.1007/978-1-4757-2440-0.

W. Patrick Walters. We need better benchmarks for machine learning in drug discovery. Practical Cheminformatics Blog, 2023. URL http://practicalcheminformatics.blogspot.com/2023/08/we-need-better-benchmarks-for-machine.html. Accessed: 2026-04-12.

David Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988.

Antony J. Williams, Christopher M. Grulke, and Jeff et al. Edwards. The comptox chemistry dashboard: a community data resource for environmental chemistry. *Journal of Cheminformatics*, 9:61, 2017. doi: 10.1186/s13321-017-0247-6.

Cas Wognum et al. Polaris: A benchmarking platform for drug discovery, 2024. URL https://doi.org/10.5281/zenodo.13652587.

D.H. Wolpert and W.G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi: 10.1109/4235.585893.

Zhenqin Wu, Bharath Ramsundar, Evan N. Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S. Pappu, Karl Leswing, and Vijay Pande. Moleculenet: A benchmark for molecular machine learning, 2018.

Liangxu Xie, Lei Xu, Ren Kong, Shan Chang, and Xiaojun Xu. Improvement of prediction performance with conjoint molecular fingerprint in deep learning. *Frontiers in Pharmacology*, 11, 2020. doi: 10.3389/fphar.2020.606668.

Yilun Xu, Peng Cao, Yuqing Kong, and Yizhou Wang. L_dmi: An information-theoretic noise-robust loss function, 2019. arXiv:1909.03388 [cs.LG].

Ge Yan, Huaijin Wu, and Junchi Yan. Quantum 3D graph learning with applications to molecule embedding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 39126–39137. PMLR, 2023.

Chu-I Yang and Yi-Pei Li. Explainable uncertainty quantifications for deep learning-based molecular property prediction. *Journal of Cheminformatics*, 15, 02 2023. doi: 10.1186/s13321-023-00682-3.

Douglas Young, Todd Martin, Raghuraman Venkatapathy, and Phil Harten. Are the chemical structures in your QSAR correct? *QSAR & Combinatorial Science*, 27(11-12):1337–1345, 2008. doi: 10.1002/qsar.200810084.

Barbara Zdrazil, Eloy Felix, Fiona Hunter, et al. The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192, 11 2023. ISSN 0305-1048. doi: 10.1093/nar/gkad1004.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

Ji-Hu Zhang, Thomas D. Y. Chung, and Kevin R. Oldenburg. A simple statistical parameter for use in evaluation and validation of high throughput screening assays. *Journal of Biomolecular Screening*, 4(2):67–73, 1999. doi: 10.1177/108705719900400206.

Zixiao Zhao, Fatemeh H. Fard, and Jie JW Wu. Mantra: a framework for multi-stage adaptive noise treatment during training, 2025.